# บทที่ 2

# HLA Fundamental And Architecture




© Zyntelligent Co., Ltd.

# HLA Fundamental and Architecture

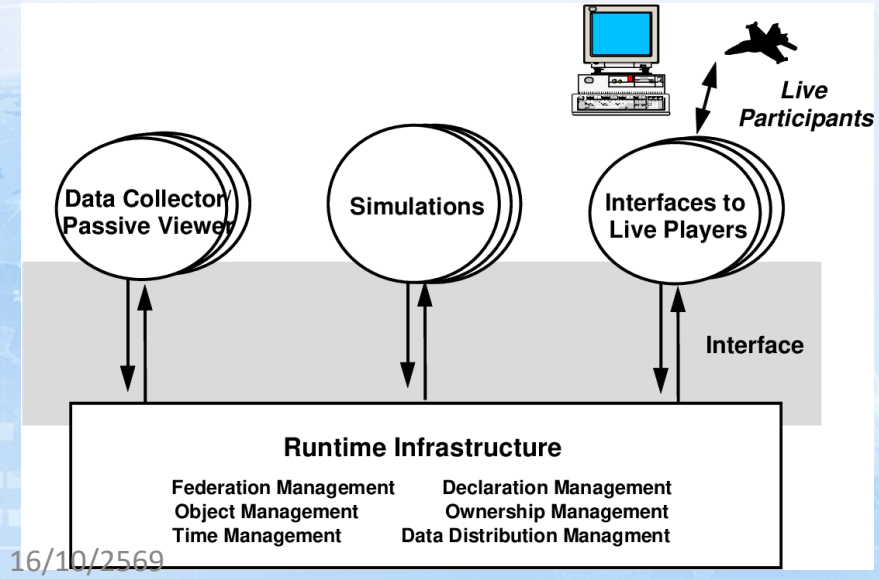
- **High Level Architecture (HLA) คืออะไร**
- องค์ประกอบหลักใน HLA
- ทิศทางการสื่อสาร RTI Service
- ภาพรวม RTI Service หลัก
- ลำดับการทำงานในการเขียน Federate
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# High Level Architecture (HLA)

**HLA หรือ High Level Architecture** คือ มาตรฐานโครงสร้างสถาปัตยกรรมระดับสากลที่ถูกกำหนดขึ้น เพื่อใช้ในการบูรณาการระบบจำลอง (Simulation) หรือระบบจำลองยุทธ์ที่หลากหลายและต่างแพลตฟอร์มกันให้สามารถเชื่อมต่อ แลกเปลี่ยนข้อมูล และทำงานร่วมกันภายในสภาวะแวดล้อมจำลองเดียวกันได้







© Zyntelligent Co., Ltd.

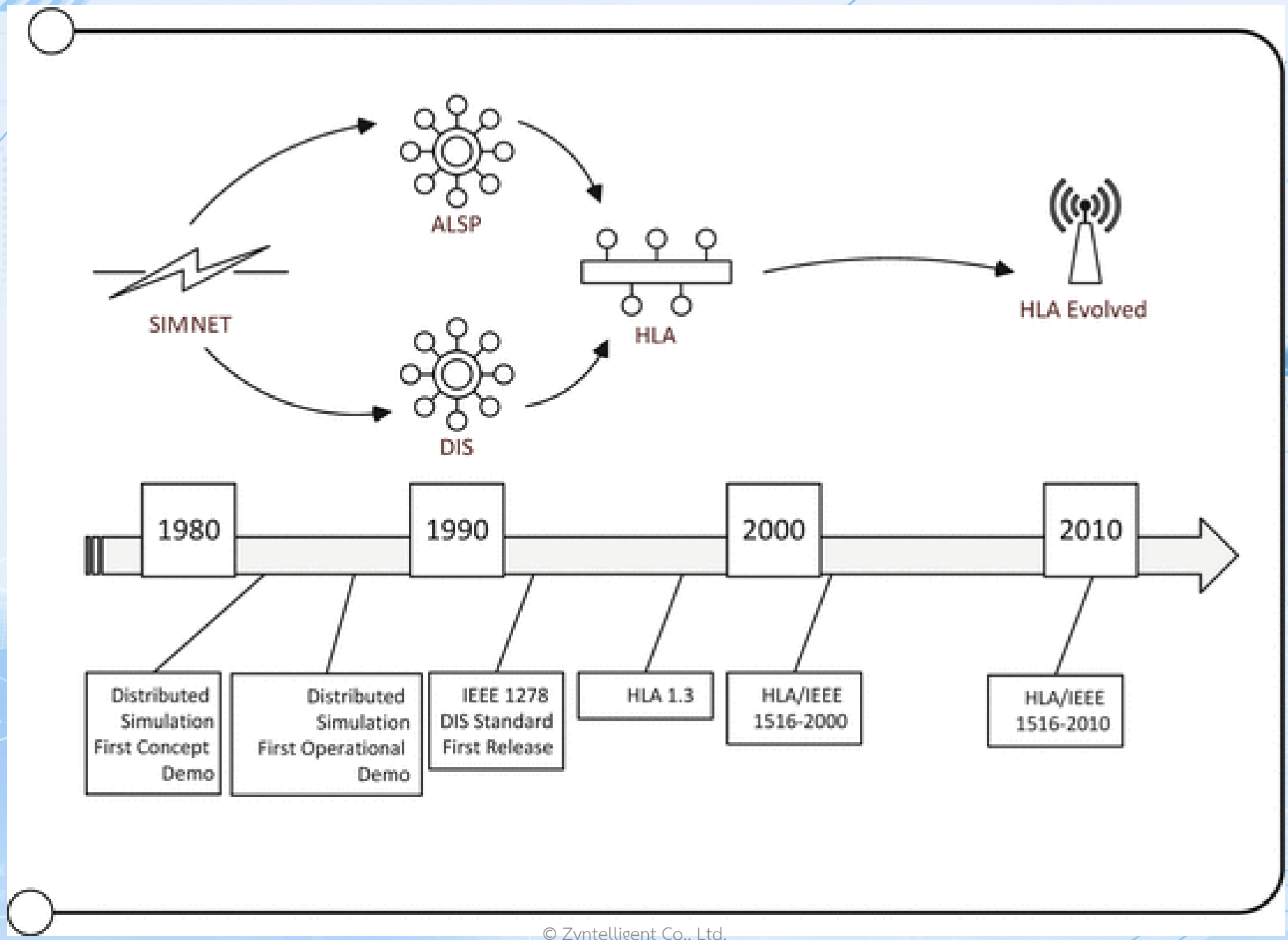
# High Level Architecture (HLA)

HLA มีโครงสร้างสถาปัตยกรรมส่วนกลาง ที่กำหนดกฎเกณฑ์และมาตรฐานการสื่อสาร ทำให้ Simulator แต่ละตัว สามารถส่งผ่านข้อมูลเข้าหากันได้อย่าง Real-time และมีประสิทธิภาพ ก่อให้เกิดเป็น **Joint Simulation**

**Environment**




© Zyntelligent Co., Ltd.





© Zyntelligent Co., Ltd.

# DIS → HLA

## DIS
(Distributed Interactive Simulation)

DIS เป็นมาตรฐานตั้งเดิมสำหรับการจำลองและวิเคราะห์ของกระทรวงกลาโหมสหรัฐฯ

### ข้อจำกัดของ DIS

- ออกแบบมาสำหรับโลกจำลองเท่านั้น
- ไม่มีแนวคิดเรื่องเวลาของระบบจำลอง
- ทุกอย่างเป็นแบบเปิด (Open Loop)



## HLA
(High Level Architecture)

HLA จึงได้รับการออกแบบโดย DMSO และ MIT ในช่วงต้น-กลางทศวรรษ 1990 โดยมีจุดประสงค์เพื่อเป็นมาตรฐานสำหรับการจำลองและวิเคราะห์ของกระทรวงกลาโหมสหรัฐฯ แทนที่ DIS





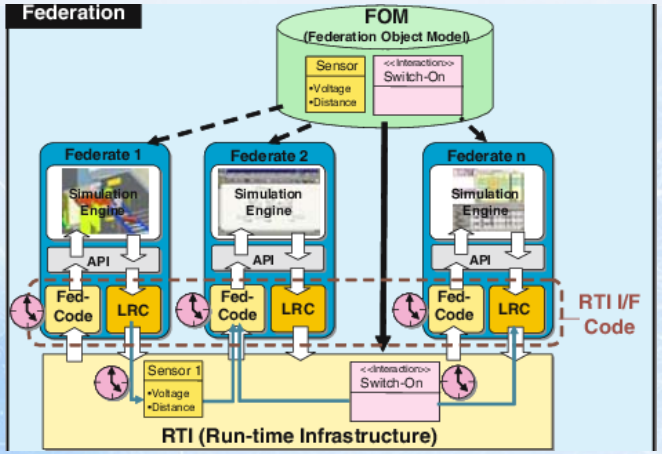
© Zyntelligent Co., Ltd.

# Use Case

Use case ที่พบบ่อยที่สุดสำหรับ HLA ได้แก่

- การฝึกอบรม ซึ่งเป็นการฝึกให้มนุษย์ปฏิบัติงาน
- การวิเคราะห์ ซึ่งเราสามารถทดลองสถานการณ์ต่างๆ ในโลกจำลองได้
- การทดสอบ วิศวกรรม การจำลอง
- ฯลฯ

แนวคิด HLA ได้รับการพัฒนาขึ้นครั้งแรกสำหรับการใช้งานด้านการป้องกันประเทศ ซึ่งยังคงเป็น Use Case หลักที่สำคัญสำหรับ HLA







© Zyntelligent Co., Ltd.

# HLA Rule 10 ข้อ

## Federation Rules

**กฎข้อที่** 1: Federations shall have an HLA Federation Object Model (FOM), documented in accordance with the HLA Object Model Template (OMT).

> **การมีอยู่ของแม่แบบจำลองวัตถุระดับภาพรวม** (FOM)

**กฎข้อที่** 2: In a federation, all simulation-associated object instance representation shall be in the federates, not in the runtime infrastructure (RTI).

> **การจำลองคุณลักษณะของวัตถุต้องอยู่ฝั่ง** Federate **เท่านั้น**

**กฎข้อที่** 3: During a federation execution, all exchange of FOM data among joined federates shall occur via the RTI.

> **การแลกเปลี่ยนข้อมูล** FOM **ทั้งหมดต้องผ่าน** RTI **เท่านั้น**

**กฎข้อที่** 4: During a federation execution, joined federates shall interact with the RTI in accordance with the HLA interface specification.

> **การปฏิสัมพันธ์กับ** RTI **ต้องเป็นไปตามข้อกำหนดอินเทอร์เฟซมาตรฐาน**

**กฎข้อที่** 5: During a federation execution, an instance attribute shall be owned by, at most, one joined federate at any given time.

> **แอตทริบิวต์ของวัตถุมีเจ้าของได้เพียงผู้เดียวในเสี้ยวเวลาใดเวลาหนึ่ง**


© Zyntelligent Co., Ltd.

# HLA Rule 10 ข้อ

## Federate Rules

**กฎข้อที่** 6: Federates shall have an HLA Simulation Object Model (SOM), documented in accordance with the HLA Object Model Template (OMT)

> **การมีอยู่ของแม่แบบจำลองวัตถุระดับระบบย่อย** (SOM)

**กฎข้อที่** 7: Federates shall be able to update and/or reflect any instance attributes and send and/or receive interactions, as specified in their SOMs.

> **ความสามารถในการอัปเดตและรับรู้คุณลักษณะตามที่ระบุใน** SOM

**กฎข้อที่** 8: Federates shall be able to transfer and/or accept ownership of instance attributes dynamically during a federation execution, as specified in their SOMs

> **ความสามารถในการโอนย้ายและยอมรับสิทธิ์ในแอตทริบิวต์แบบไดนามิก**

**กฎข้อที่** 9: Federates shall be able to vary the conditions (e.g., thresholds) under which they provide updates of instance attributes, as specified in their SOMs.

> **ความสามารถในการปรับเปลี่ยนเงื่อนไขในการส่งอัปเดตข้อมูล**

**กฎข้อที่** 10: Federates shall be able to manage local time in a way that will allow them to coordinate data exchange with other members of a federation.

> **ความสามารถในการจัดการเวลาระดับท้องถิ่นเพื่อผูกประสานข้อมูล**


© Zyntelligent Co., Ltd.

# HLA Fundamental and Architecture

- High Level Architecture (HLA) คืออะไร
- **องค์ประกอบหลักใน HLA**
- ทิศทางการสื่อสาร RTI Service
- ภาพรวม RTI Service หลัก
- ลำดับการทำงานในการเขียน Federate
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

Federation

FOM

Object Class

Parameter

Federate

SOM

Attribute

RTI

OMT

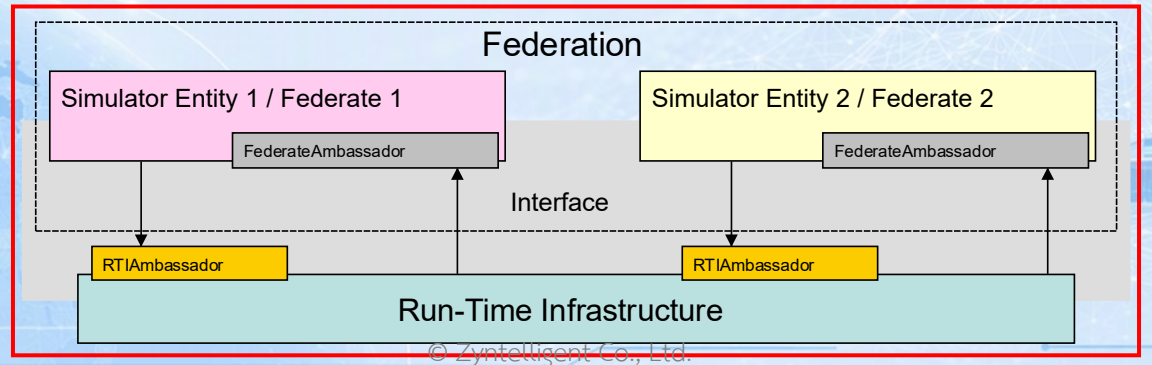
Interaction Class


© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## Federation

- ระบบจำลองรวมที่มีหลาย Federate ทำงานร่วมกันภายใต้กติกาเดียวกัน
- รวบรวม Federate หลายๆ ตัวเข้าด้วยกันให้ทำงานเป็นระบบเดียว
- กำหนดขอบเขตของการจำลอง เช่น การซ้อมรบทางทหารที่มี simulator เครื่องบิน, เรือ, radar มาทำงานร่วมกัน
- ผูกพันด้วย FOM เดียวกัน ทุก Federate ในนี้ต้องสื่อสารกันด้วยข้อมูลตามที่กำหนดไว้ใน FOM
- Federation created ⟶ Federate join ⟶ Run ⟶ Federate Resign ⟶ Federation destroy





© Zyntelligent Co., Ltd.

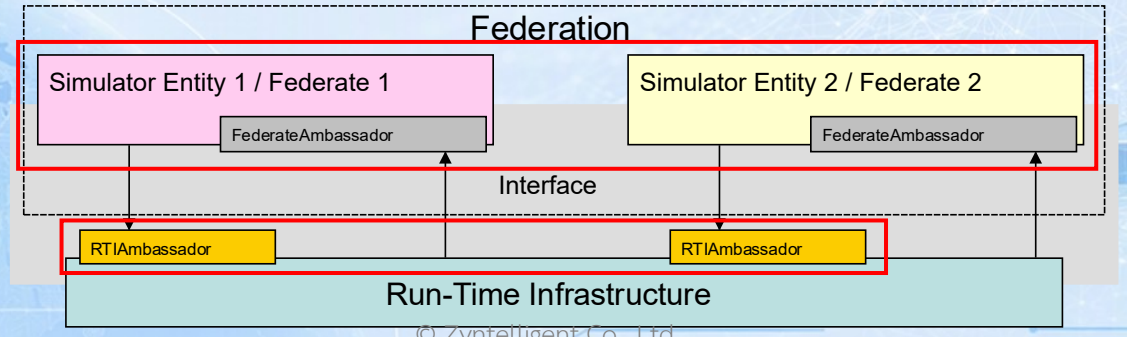
# องค์ประกอบหลักใน HLA

## Federate

- โปรแกรมจำลองหรือองค์ประกอบหนึ่งตัว เช่น Truck, Tank, Aircraft เป็นต้น
- จำลองพฤติกรรมหรือระบบเฉพาะของตัวเอง เช่น Federate หนึ่งจำลองเครื่องบิน อีกตัวจำลองสภาพอากาศ
- ส่งข้อมูล (Publish) สถานะของตัวเองให้ Federate อื่นที่อยู่ใน Federation เดียวกันรับรู้
- รับข้อมูล (Subscribe) จาก Federate อื่นเพื่อนำไปใช้ในการจำลองของตน
- Federate จะติดต่อกับ RTI เท่านั้น โดยจะไม่ติดต่อกับ Federate อื่นโดยตรง ซึ่งจะติดตั้ง ผ่าน 2 interface หลัก ดังนี้:

  **RTI Ambassador**: ช่องทางที่ Federate จะสั่งงาน RTI (เช่น ขอสร้าง object, ส่ง interaction)

  **Federate Ambassador**: ช่องทางที่ RTI จะแจ้งเตือนกลับมาที่ Federate (callback)



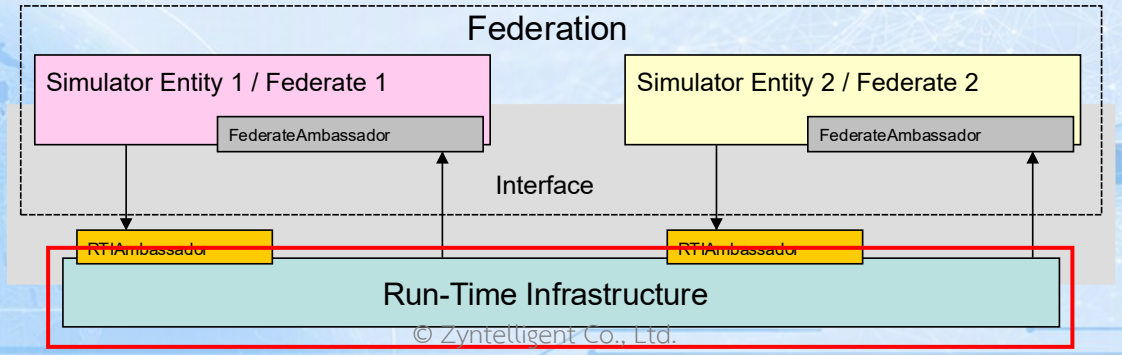


© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## RTI (Run-Time Infrastructure)

- Runtime Infrastructure ทำหน้าที่เป็น Middleware สำหรับ จัดการและให้บริการต่างๆ แก่ Federate ตามข้อกำหนด
- ส่งต่อข้อมูล (Object update, Interaction) ระหว่าง Federate อย่างถูกต้องและทันเวลา
- จัดการ Publish/Subscribe รู้ว่าใครสนใจข้อมูลอะไร แล้วส่งเฉพาะที่เกี่ยวข้อง
- ควบคุม Time Management ให้เวลาของทุก Federate เดินสอดคล้องกัน
- จัดการ Ownership ของ Attribute
- ทำหน้าที่เป็นตัวกลาง ไม่ให้ Federate ใดควบคุมระบบทั้งหมดโดยตรง





© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## FOM (Federation Object Model)

- โมเดลข้อมูลร่วมของทั้ง Federation อธิบายว่ามี Object/Interaction อะไรบ้างที่แชร์กัน
- กำหนดว่ามี Object Class และ Interaction Class อะไรบ้างที่แชร์กันใน Federation
- เป็น contract ที่ทำให้ Federate ต่างระบบ ต่างผู้พัฒนา สามารถเข้าใจข้อมูลของกันและกันได้
- ถูกโหลดเข้า RTI ตอนสร้าง Federation เพื่อกำหนดกรอบการสื่อสารกันภายใน Federation

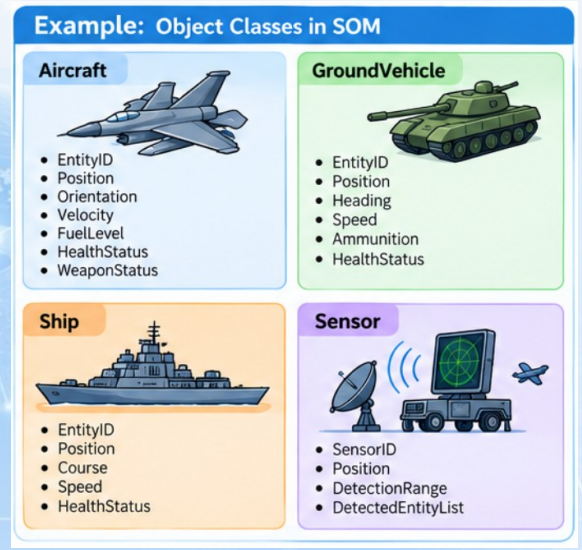
```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<objectModel xsi:schemaLocation="http://standards.ieee.org/IEEE1516-2010 http://standards.ieee.org/d
    <modelIdentification notes="Note2">
        <name>Serving processes for Restaurant FOM</name>
        <type>FOM</type>
        <version>3.0</version>
        <modificationDate>2010-09-09</modificationDate>
        <securityClassification>Unclassified</securityClassification>
        <releaseRestriction>Other organizations not related to Federated Foods</releaseRestriction>
        <purpose>To define a processes module for a restaurant federation</purpose>
        <applicationDomain>Restaurant operations</applicationDomain>
        <description>Processes for the modular restaurant.</description>
        <useLimitation>Derived models must reference this document</useLimitation>
        <useHistory>Originally used as an example in the DoD HLA 1.3 specification</useHistory>
        <useHistory>Used as an example in the HLA IEEE1516.2-2000 specification</useHistory>
        <useHistory>Used as an example in the HLA IEEE1516.2-2010 specification after conversion to
        <keyword>
```

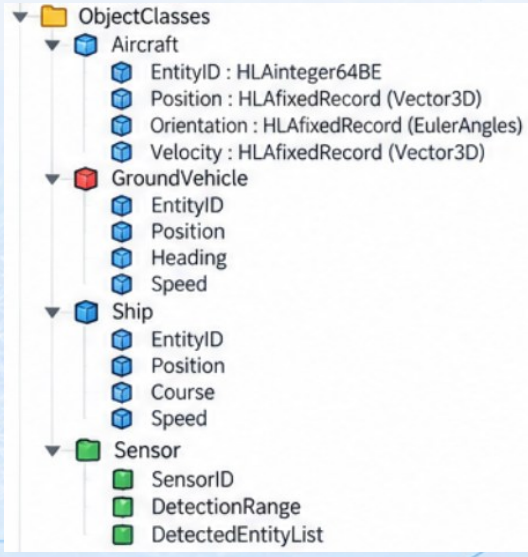

© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## SOM (Simulation Object Model)

- โมเดลข้อมูลของ Federate แต่ละตัว อธิบายความสามารถที่ Federate นั้นมี
- อธิบายว่า Federate ตัวนี้สามารถ Publish/Subscribe อะไรได้บ้าง (จากขีดความสามารถของตัวเอง)
- ใช้เป็นข้อมูลอ้างอิงตอนออกแบบ FOM ของ Federation ว่าจะรวมความสามารถของแต่ละ Federate อย่างไร
- ช่วยให้ Federate ตัวเดิมนำไปใช้ใน Federation อื่นได้ (Reusability) เพราะ SOM อธิบายตัวเองไว้ชัดเจนแล้ว



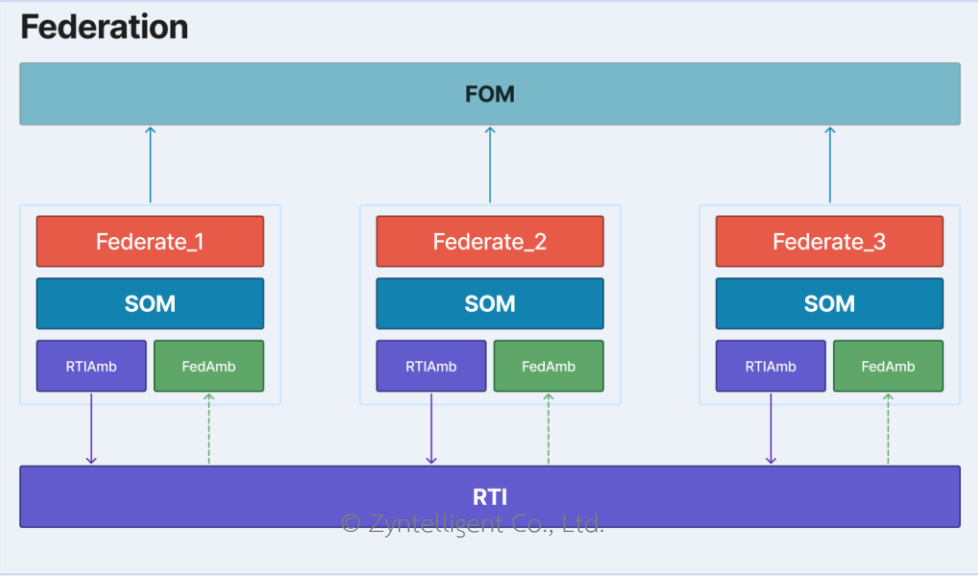


© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## OMT (Object Model Template)

- รูปแบบมาตรฐาน ที่ใช้เขียน FOM/SOM ให้อยู่ในรูปแบบเดียวกัน
- กำหนดโครงสร้าง, ตาราง, และรูปแบบเอกสารตามมาตฐานที่ใช้เขียน Object Model
- ทำให้ FOM/SOM จากผู้พัฒนาต่างค่าย ต่างเครื่องมือ อ่านและเข้าใจตรงกันได้ เหมือนใช้ template เดียวกัน
- ปัจจุบันนิยมเขียนในรูปแบบไฟล์ XML (FDD - FOM Document Data)





© Zyntelligent Co., Ltd.

# องค์ประกอบหลักใน HLA

## Object Class

- Object Class ที่มีสถานะคงอยู่ตลอดเวลา (persistent state) เช่น Aircraft

## Attribute

- คุณสมบัติของ Object Class นั้น เช่น Position, Speed, Health เป็นค่าที่เปลี่ยนแปลงได้และถูกอัปเดตส่งไปมาระหว่าง Federate

## Interaction Class

- เหตุการณ์ที่เกิดขึ้น ณ จุดเวลาหนึ่ง เช่น WeaponFire, Collision, MessageSent

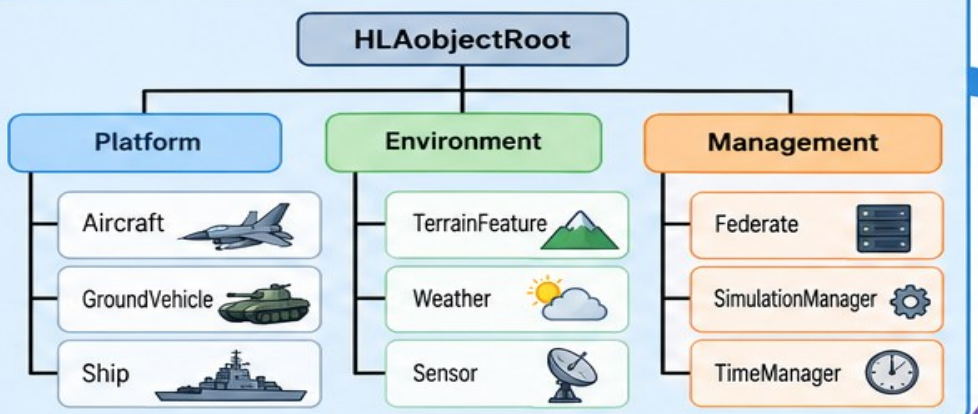
## Parameter

- ข้อมูลที่แนบไปกับเหตุการณ์นั้น เช่น ทิศทาง, ความแรง, ผู้ส่ง

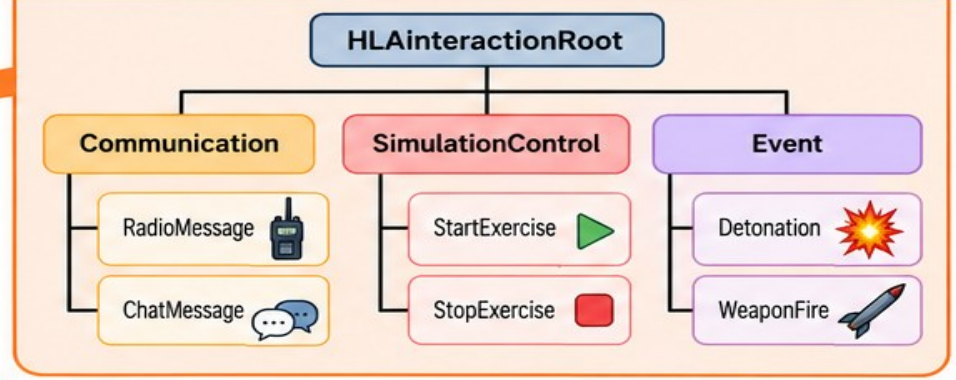

© Zyntelligent Co., Ltd.

# Federation Object Model (FOM)
## in HLA IEEE 1516.2010

### Object Class




### Interaction Class




### FOM (XML)

```
<ObjectClass>
  ...
</objectClass>
<interactionClass>
  ...
</interactionClass>
<dataType>
  ...
</dataType>
```

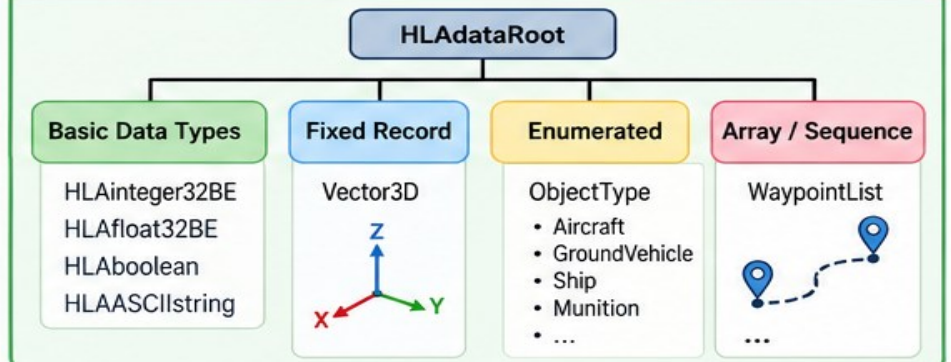
### Attributes (ของแต่ละ Object Class)

**Aircraft**



| Attribute Name | Data Type |
|---|---|
| EntityID | HLAinteger64BE |
| Position | HLAfixedRecord (Vector3D) |
| Orientation | HLAfixedRecord (EulerAngles) |
| Velocity | HLAfixedRecord (Vector3D) |
| FuelLevel | HLAfloat32BE |
| HealthStatus | HLAenumerated (Operational, Damaged, ...) |

### Data Type




### Federation Execution (RTI)





© Zyntelligent Co., Ltd.

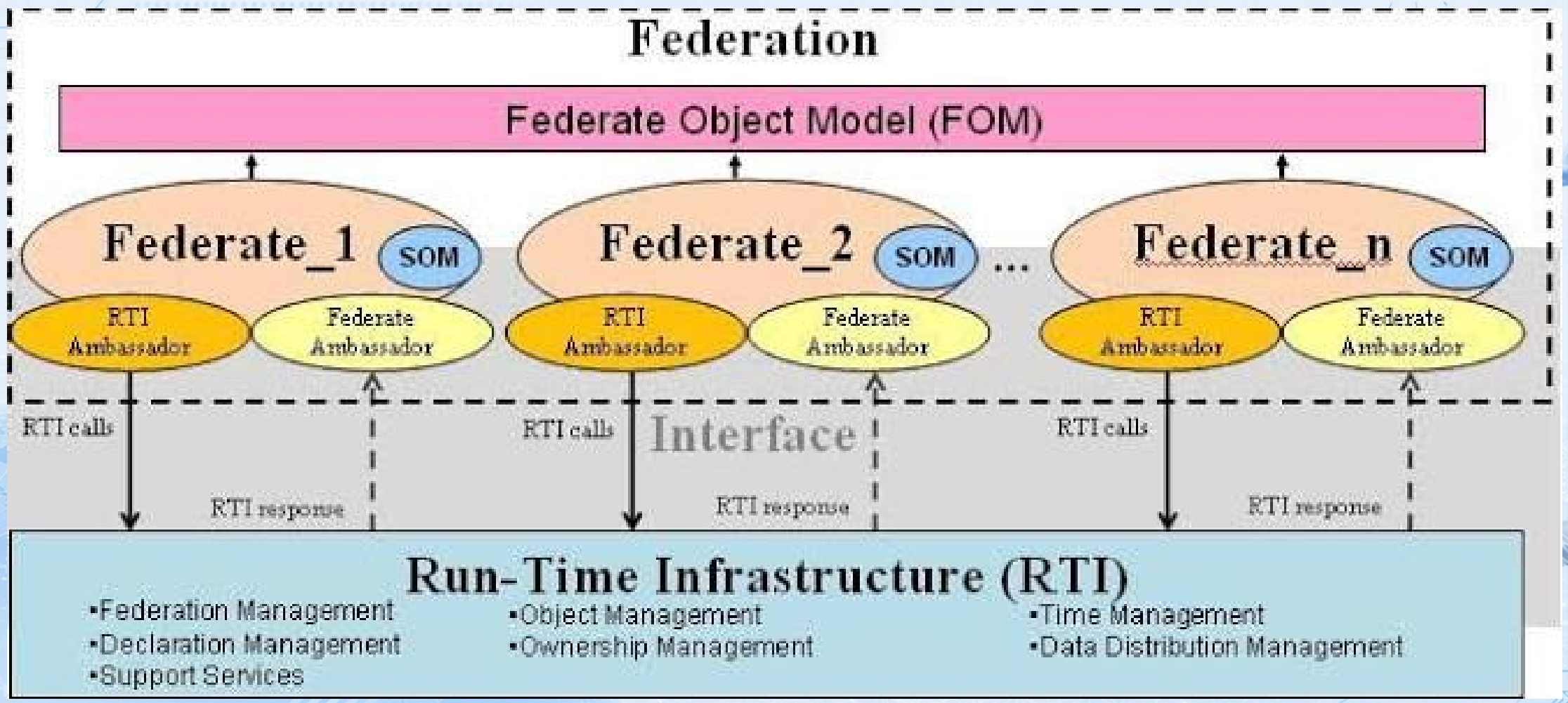
# HLA Fundamental and Architecture

- High Level Architecture (HLA) คืออะไร
- องค์ประกอบหลักใน HLA
- ทิศทางการสื่อสาร RTI Service
- ภาพรวม RTI Service หลัก
- ลำดับการทำงานในการเขียน Federate
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# ทิศทางการสื่อสาร RTI Service





© Zyntelligent Co., Ltd.

# HLA Fundamental and Architecture

- High Level Architecture (HLA) คืออะไร
- องค์ประกอบหลักใน HLA
- ทิศทางการสื่อสาร RTI Service
- **ภาพรวม RTI Service หลัก**
- ลำดับการทำงานในการเขียน Federate
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

- Federation Management
- Ownership Management
- Declaration Management
- Time Management
- Object Management
- Data Distribution Management
- Support Services
- Management Object Model (MOM)




© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 1. Federation Management

- **Create Federation Execution**: สร้าง Federation ขึ้นมาใหม่ พร้อมโหลด FOM
- **Join Federation Execution**: ให้ Federate เข้าร่วม Federation ที่มีอยู่
- **Resign Federation Execution**: ให้ Federate ออกจาก Federation
- **Destroy Federation Execution**: ทำลาย Federation เมื่อไม่มี Federate เหลืออยู่
- **Request Federation Save:** จัดการ Save/Restore สถานะของ Federation

และ อื่น ๆ


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 2. Declaration Management

- **Publish**: Federate ประกาศว่าจะส่งข้อมูล Object Class หรือ Interaction Class ใด
- **Subscribe**: Federate ประกาศว่าต้องการรับข้อมูลประเภทใด
- **Unpublish:** Federate ยกเลิกการส่งข้อมูล Object Class หรือ Interaction Class
- **Unsubscribe:** Federate ยกเลิกการรับข้อมูล Object Class หรือ Interaction Class


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 3. Object Management

- **Register Object Instance:** สร้าง instance ของ Object Class ขึ้นมาจริงในระบบ
- **Update Attribute Values:** ส่งค่าล่าสุดของ Attribute ไปให้ Federate ที่ subscribe
- **Send Interaction:** ส่งเหตุการณ์ (Interaction) พร้อม Parameter ไปยัง Federate ที่เกี่ยวข้อง
- **Delete Object Instance:** ลบ Object instance เมื่อไม่ใช้แล้ว เช่น เครื่องบินถูกยิงตก
- **Reflect/Receive:** ฝั่งผู้รับจะได้ callback เมื่อมีการ update/interaction เกิดขึ้น

และ อื่น ๆ


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 4. Ownership Management

- Attribute Ownership
- Request Attribute Ownership Assumption
- กำหนดว่า ณ เวลาหนึ่ง มีเพียง Federate เดียวเท่านั้นที่เป็นเจ้าของและแก้ไขได้ Attribute หนึ่ง
- รองรับการโอนสิทธิ์ (Ownership Transfer) ระหว่าง Federate เช่น เมื่อ object เปลี่ยนการควบคุมจาก Federate หนึ่งไปอีกตัว
- ป้องกันปัญหา "สองคนเขียนพร้อมกัน" (write conflict)

และ อื่น ๆ


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 5. Time Management

- **Enable Time Regulation**
- **Enable Time Constrained**
- **Next Message Request**
- รักษาความถูกต้องเชิงเวลา (Causality) เหตุการณ์ต้องเกิดตามลำดับเวลาที่ถูกต้อง ไม่สลับก่อนหลัง
- จัดการ Federate ที่เป็น Time Regulating (ควบคุมจังหวะเวลา) และ Time Constrained (ต้องรอตามเวลา)
- ใช้แนวคิด Lookahead (ระยะเวลาที่ Federate การันตีว่าจะไม่ส่งเหตุการณ์ย้อนเวลา) เพื่อให้ RTI ประมวลผลแบบ Time-Stepped หรือ Event-Driven ได้อย่างถูกต้อง
- รองรับทั้งโหมด TSO (Time Stamp Order) และ RO (Receive Order)


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## 6. Data Distribution Management (DDM)

- ใช้แนวคิด Region (พื้นที่ที่สนใจ) แทนการ subscribe แบบกว้างๆ ทั้ง Class
- ส่งข้อมูลเฉพาะให้ Federate ที่อยู่ใน "พื้นที่หรือขอบเขตที่เกี่ยวข้อง" เท่านั้น ไม่ส่งทุกอย่างให้ทุกคน
- สำคัญมากในระบบขนาดใหญ่ เช่น การจำลองสมรภูมิที่มีวัตถุนับพันชิ้น — ไม่ต้องการให้ทุก Federate รับข้อมูลของวัตถุที่อยู่ไกลเกินไป


© Zyntelligent Co., Ltd.

# ภาพรวม RTI Service

## Services อื่น ๆ

### Support Services

- เป็น Service ที่ช่วยสนับสนุนการทำงาน หรือ Utilities Class

### Management Object Model (MOM)

- ใช้สำหรับเป็นเครื่องมือ Debug/Monitor


© Zyntelligent Co., Ltd.

# HLA Fundamental and Architecture

- High Level Architecture (HLA) คืออะไร
- องค์ประกอบหลักใน HLA
- ทิศทางการสื่อสาร RTI Service
- ภาพรวม RTI Service หลัก
- **ลำดับการทำงานในการเขียน Federate**
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# ลำดับการทำงาน

1.Create the RTIambassador

2.Create the Federation (skip if one exists)

3.Join the Federation

4.Initialize all handles

5.Announce a Synchronization Point

6.Wait for the user to press Enter

7.Achieve the Synchronization Point

8.Enable federate's time settings

9. Publish and Subscribe

10. Register an Object Instance

11. Run Federate/Simulator Tasks

12. Delete the object instance

13. Resign from the Federation

14. Destroy the Federation (if possible)

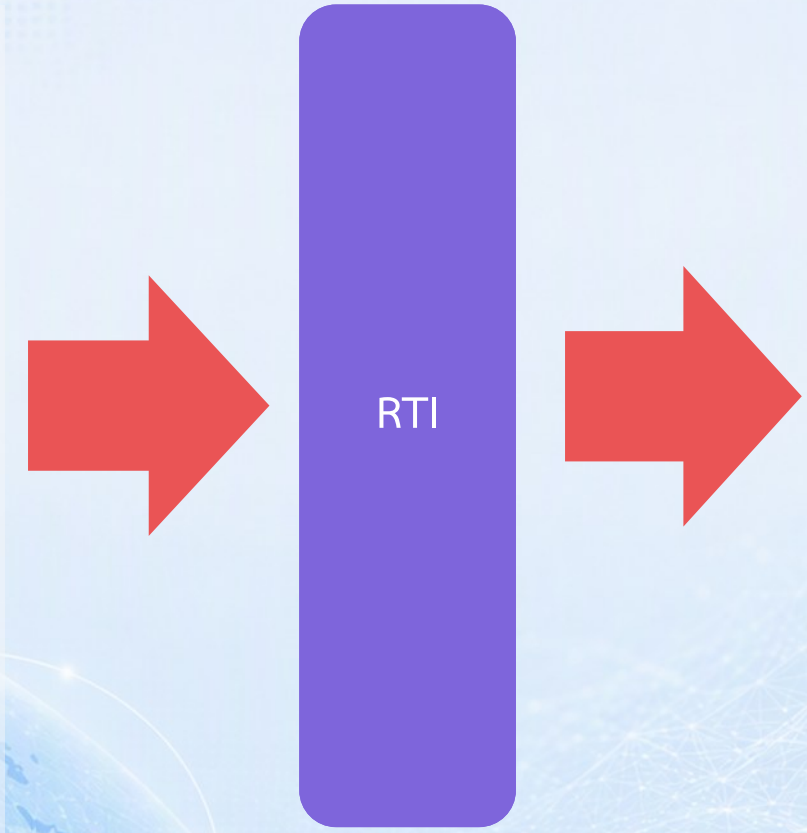



© Zyntelligent Co., Ltd.

# ลำดับการทำงาน

**Federate.java**

- connect()
- createFederationExecution()
- joinFederationExecution()
- registerSyncPoint()
- synchronizationPointAchieved()
- enableTimeRegulation()
- publish/subscribe()
- registerObjectInstance()
- updateAttributeValues()
- sendInteraction()
- timeAdvanceRequest()
- deleteObjectInstance()
- resign / destroy




**FederateAmbassador.java**

- announceSynchronizationPoint()
- federationSynchronized()
- timeRegulationEnabled()

- discoverObjectInstance() [ที่ federate อื่น]
- reflectAttributeValues() [ที่ federate อื่น]
- receiveInteraction() [ที่ federate อื่น]
- timeAdvanceGrant()
- removeObjectInstance()


© Zyntelligent Co., Ltd.

# HLA Fundamental and Architecture

- High Level Architecture (HLA) คืออะไร
- องค์ประกอบหลักใน HLA
- ทิศทางการสื่อสาร RTI Service
- ภาพรวม RTI Service หลัก
- ลำดับการทำงานในการเขียน Federate
- การออกแบบ Methods ใน Federate
- ตัวอย่างการเขียน Methods ใน Federate




© Zyntelligent Co., Ltd.

# Methods Design

## Federate.java

```
// ---------- STATIC ----------
public static final String READY_TO_RUN = "ReadyToRun";

// ---------- FIELDS ----------
private RTIambassador rtiamb;
private ExampleFederateAmbassador fedamb;
private HLAfloat64TimeFactory timeFactory;
protected EncoderFactory encoderFactory;

protected ObjectClassHandle      sodaHandle;
protected AttributeHandle        cupsHandle, flavHandle;
protected InteractionClassHandle servedHandle;


// ---------- LIFECYCLE ----------
public void runFederate(String federateName)
private void enableTimePolicy()
private void publishAndSubscribe()
```

```
// ---------- OBJECT & INTERACTION ----------
private ObjectInstanceHandle registerObject()
private void updateAttributeValues(ObjectInstanceHandle handle)
private void sendInteraction()
private void deleteObject(ObjectInstanceHandle handle)

// ---------- TIME ----------
private void advanceTime(double timestep)

// ---------- UTILITY ----------
private void log(String msg)
private void waitForUser()
private short getTimeAsShort()
private byte[] generateTag()

public static void main(String[] args)
```


© Zyntelligent Co., Ltd.

# Methods Design

## Federate.java

```
public void run()
{
    try
    {
        log( "=======================================" );
        log( " Federate: " + federateName + "" );
        log( "=======================================" );

        prepareRTI();
        joinFederation();
        waitForUser();
        publishAndSubscribe();
        enableTimePolicy();
        synchronize( Constants.SYNC_READY_TO_RUN );
        registerObjects();
        runSimulation();
        synchronize( Constants.SYNC_READY_TO_RESIGN );
        resignAndCleanup();

        log( "=======================================" );
        log( " Federate " + federateName + " terminated." );
        log( "=======================================" );
    }
    finally
    {
        cleanup();
    }
}
```


© Zyntelligent Co., Ltd.

# Methods Design

## FederateAmbassador.java

```java
// STATE
protected double  federateTime      = 0.0;
protected double  federateLookahead = 1.0;

protected boolean isRegulating  = false;
protected boolean isConstrained = false;
protected boolean isAdvancing   = false;
protected boolean isAnnounced   = false;
protected boolean isReadyToRun  = false;


public ExampleFederateAmbassador(ExampleFederate federate)

private void log(String message)

// FEDERATION MANAGEMENT CALLBACKS
public void synchronizationPointRegistrationSucceeded(String label)
public void synchronizationPointRegistrationFailed(String label, SynchronizationPointFailureReason reason)
public void announceSynchronizationPoint(String label, byte[] tag)
public void federationSynchronized(String label, FederateHandleSet failed)


// TIME MANAGEMENT CALLBACKS
public void timeRegulationEnabled(LogicalTime time)
public void timeConstrainedEnabled(LogicalTime time)
public void timeAdvanceGrant(LogicalTime time)
```

```java
// OBJECT MANAGEMENT CALLBACKS
public void discoverObjectInstance(ObjectInstanceHandle theObject,
                                   ObjectClassHandle theObjectClass,
                                   String objectName)

public void reflectAttributeValues(ObjectInstanceHandle theObject,
                                   AttributeHandleValueMap theAttributes,
                                   byte[] tag,
                                   OrderType sentOrder,
                                   TransportationTypeHandle transport,
                                   SupplementalReflectInfo reflectInfo)

public void reflectAttributeValues(ObjectInstanceHandle theObject,
                                   AttributeHandleValueMap theAttributes,
                                   byte[] tag,
                                   OrderType sentOrder,
                                   TransportationTypeHandle transport,
                                   LogicalTime time,
                                   OrderType receivedOrder,
                                   SupplementalReflectInfo reflectInfo)

public void receiveInteraction(InteractionClassHandle interactionClass,
                               ParameterHandleValueMap theParameters,
                               byte[] tag,
                               OrderType sentOrdering,
                               TransportationTypeHandle theTransport,
                               SupplementalReceiveInfo receiveInfo)

public void receiveInteraction(InteractionClassHandle interactionClass,
                               ParameterHandleValueMap theParameters,
                               byte[] tag,
                               OrderType sentOrdering,
                               TransportationTypeHandle theTransport,
                               LogicalTime time,
                               OrderType receivedOrdering,
                               SupplementalReceiveInfo receiveInfo)

public void removeObjectInstance(ObjectInstanceHandle theObject,
                                 byte[] tag,
                                 OrderType sentOrdering,
                                 SupplementalRemoveInfo removeInfo)
```


© Zyntelligent Co., Ltd.

# prepareRTI()

```
private void prepareRTI() throws RTIexception
    {
        log( ">> 1. Prepare RTI..." );

        RtiFactory factory = RtiFactoryFactory.getRtiFactory();
        rtiamb = factory.getRtiAmbassador();
        encoderFactory = factory.getEncoderFactory();
        fedamb = new ScenarioExampleFederateAmbassador( encoderFactory );
        timeFactory = (HLAfloat64TimeFactory)rtiamb.getTimeFactory();

        log( "   RTIambassador created." );
        log( "   RTI: " + factory.rtiName() + " v" + factory.rtiVersion() );
    }
```



สร้าง factory และ object หลักที่ใช้คุยกับ RTI เชื่อม Federate เข้ากับ RTI พร้อมรับ callback

# joinFederation()

```
private void joinFederation() throws Exception
    {
        log( ">> 2. Join federation: " + DeliveryConstants.FEDERATION_NAME );

        rtiamb.connect( fedamb, CallbackModel.HLA_IMMEDIATE );
        connected = true;
        log( "   Connected to RTI." );

        try
        {
            rtiamb.createFederationExecution( DeliveryConstants.FEDERATION_NAME, getCreateFomModules() );
            log( "   Federation created." );
        }
        catch( FederationExecutionAlreadyExists e )
        {
            log( "   Federation already exists." );
        }

        FederateHandle fedHandle = rtiamb.joinFederationExecution( federateName,
                                                                   federateType,
                                                                   DeliveryConstants.FEDERATION_NAME );
        joined = true;
        log( "   Joined as: " + federateName + " (handle: " + fedHandle + ")" );
    }
```



เชื่อมต่อ RTI จากนั้นสร้าง Federation หากยังไม่มี และเข้าร่วม Federation

# waitForUser()

```
private void waitForUser()
{
    log( " >>>>>>>>>> Press Enter to Continue <<<<<<<<<<" );
    BufferedReader reader = new BufferedReader( new InputStreamReader(System.in) );
    try
    {
        reader.readLine();
    }
    catch( Exception e )
    {
        log( "Error while waiting for user input: " + e.getMessage() );
        e.printStackTrace();
    }
}
```

รอรับ Input จากผู้ใช้งาน *Optional

# publishAndSubscribe()

```
private void publishAndSubscribe() throws RTIexception
    {

        this.packageHandle = rtiamb.getObjectClassHandle( "HLAobjectRoot.Package" );
        this.packageStatusHandle = rtiamb.getAttributeHandle( packageHandle, "Status" );

        AttributeHandleSet attributes = rtiamb.getAttributeHandleSetFactory().create();
        attributes.add( packageStatusHandle );

        rtiamb.publishObjectClassAttributes( packageHandle, attributes );
        rtiamb.subscribeObjectClassAttributes( packageHandle, attributes );

        String iname = "HLAinteractionRoot.DeliveryEvents.PackageCreated";
        packageCreatedHandle = rtiamb.getInteractionClassHandle( iname );

        rtiamb.publishInteractionClass( packageCreatedHandle );
        rtiamb.subscribeInteractionClass( packageCreatedHandle );
    }
```

กำหนดว่า Federate จะส่งหรือรับข้อมูลอะไรบ้าง

# enableTimePolicy()

```
private void enableTimePolicy() throws Exception
{
    HLAfloat64Interval lookahead = timeFactory.makeInterval( fedamb.federateLookahead );

    this.rtiamb.enableTimeRegulation( lookahead );

    // tick until we get the callback
    while( fedamb.isRegulating == false )
    {
        rtiamb.evokeMultipleCallbacks( 0.1, 0.2 );
    }

    this.rtiamb.enableTimeConstrained();
    // tick until we get the callback
    while( fedamb.isConstrained == false )
    {
        rtiamb.evokeMultipleCallbacks( 0.1, 0.2 );
    }
}
```



ตั้งค่าการจัดการเวลาของ federate เช่น logical time, regulation, constrained

# synchronize()

```
private void synchronize( String syncLabel ) throws Exception
{
    try
    {
        rtiamb.registerFederationSynchronizationPoint( syncLabel, new byte[0] );
        log( "   Registered sync point: " + syncLabel );
    }
    catch( Exception e )
    {
        log( "   Sync point already registered." );
    }

    while( !fedamb.isSyncPointKnown(syncLabel) )
        rtiamb.evokeMultipleCallbacks( 0.1, 0.2 );

    rtiamb.synchronizationPointAchieved( syncLabel );
    log( "   Achieved sync point: " + syncLabel );

    while( !fedamb.isSyncPointAchieved(syncLabel) )
        rtiamb.evokeMultipleCallbacks( 0.1, 0.2 );
    log( "   Synchronized: " + syncLabel );
}
```



การทำ Synchronization Point


© Zyntelligent Co., Ltd.

# registerObject()

```
private void registerObjects() throws RTIexception
{
    deliveryPersonObjectHandle = rtiamb.registerObjectInstance( deliveryPersonHandle, federateName + "_DeliveryPerson" );
    log( "   Registered DeliveryPerson: " + deliveryPersonObjectHandle );
}
```

การลงทะเบียน Object


© Zyntelligent Co., Ltd.

# runSimulation()

```
private void runSimulation() throws Exception
{
    log( ">> 7. Simulation (" + maxTimeSteps + " steps)..." );

    for( int step = 1; step <= maxTimeSteps; step++ )
    {
        advanceTime( timeStep );
        log( "   Step " + step + "/" + maxTimeSteps +
             " | t=" + String.format(format: "%.1f", fedamb.getCurrentTime()) + "s" );
    }

    log( message: "   Simulation complete." );
}
```



Method ที่ใช้ run การจำลอง


© Zyntelligent Co., Ltd.

# resignAndCleanup()

```
private void resignAndCleanup() throws RTIexception
{
    log( message: ">> 8. Resign..." );

    rtiamb.resignFederationExecution( ResignAction.DELETE_OBJECTS );
    joined = false;
    resigned = true;
    log( message: "   Resigned." );

    try
    {
        rtiamb.destroyFederationExecution( DeliveryConstants.FEDERATION_NAME );
        log( message: "   Federation destroyed." );
    }
    catch( FederationExecutionDoesNotExist e )
    {
        log( message: "   Federation already destroyed." );
    }
    catch( RTIexception e )
    {
        log( message: "   Federation not destroyed (other federates still joined)." );
    }

    rtiamb.disconnect();
    connected = false;
    log( message: "   Disconnected." );
}
```

การ resign ออกจาก Federation และ Disconect




© Zyntelligent Co., Ltd.

# บทที่ 3
# Federation Object Model (FOM)
# Simulation Object Model (SOM)


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ความหมายของ FOM SOM และ OMT
- Object, Attribute, Interaction และ Parameter
- โครงสร้าง FOM และชนิดข้อมูล
- Attribute และ Agreement ข้อมูล
- วิธีเขียน FOM/SOM - เริ่มต้นเขียน FOM ตามโจทย์ที่จะเรียน




© Zyntelligent Co., Ltd.

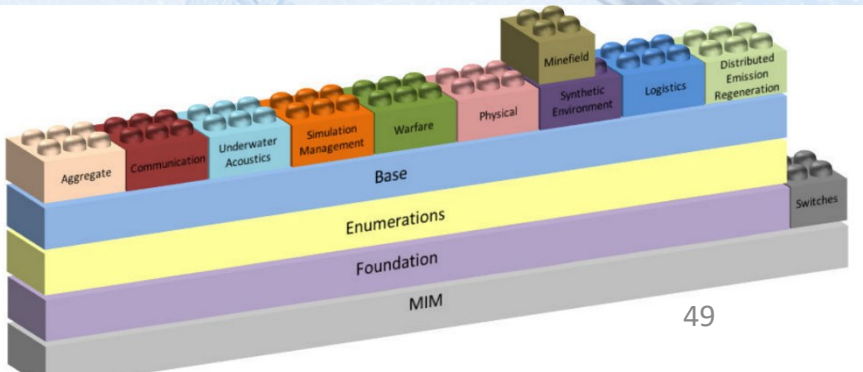
# Federation Object Model / Simulation Object Model

## Federation Object Model (FOM)

**Federation Object Model** คือ **ข้อตกลงกลางของทั้ง Federation** อธิบายข้อมูลที่สมาชิกใน Federation ใช้แลกเปลี่ยนร่วมกัน เช่นมี Class อะไร มี Attribute อะไร ใช้ชนิดข้อมูล หน่วย และความหมายใด ข้อมูลใดบ้างจะแลกเปลี่ยนกันผ่าน RTI ตาม **HLA Rules ข้อ 1**: Federation ต้องมี FOM ที่เขียนตาม OMT

RTI ใช้ FOM เพื่อ

- แปลงชื่อเป็น handle เช่น getObjectClassHandle("HLAobjectRoot.Food.Drink.Soda")
- รู้ว่า class ใดสืบทอดจาก class ใด เพื่อใช้กับ subscription แบบ class hierarchy
- รู้ transport และ order เริ่มต้นของ attribute และ interaction แต่ละตัว





© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

## Simulation Object Model (SOM)

Simulation Object Model คือ **object model** ของ Federate หนึ่งตัว ทำหน้าที่ประกาศว่า federate นั้น

- publish object class, attribute และ interaction อะไรได้บ้าง
- subscribe หรือต้องการรับข้อมูลอะไรบ้าง
- รองรับ service อะไร เช่น time management หรือ ownership transfer


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

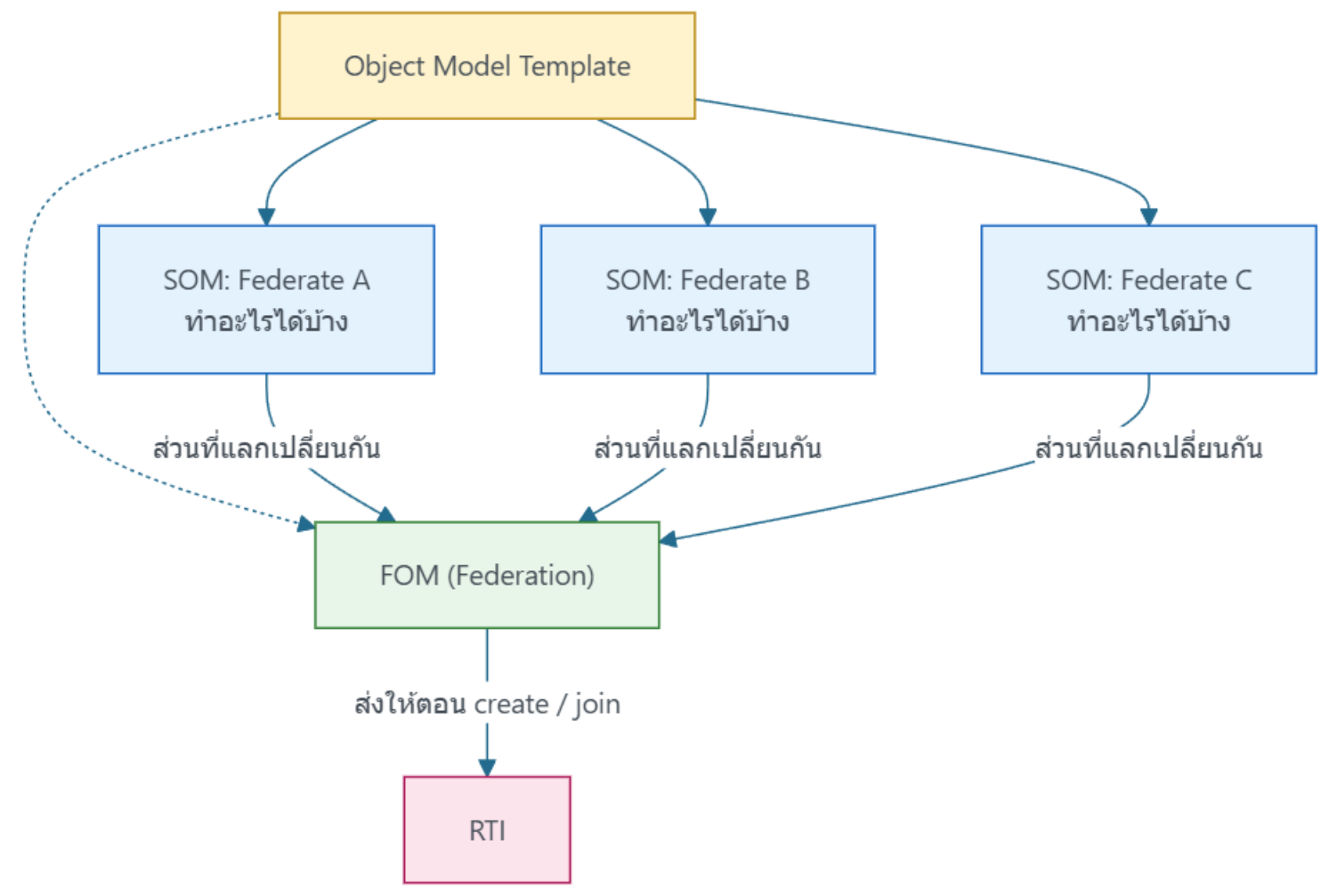
## Object Model Template (OMT)

**มาตรฐาน IEEE 1516.2-2010** จะมีสิ่งที่กำหนดว่าการอธิบาย object model ต้องมีโครงสร้าง ส่วนประกอบ และรูปแบบอย่างไร เรียกว่า Object Model Template (OMT) โดยตัว OMT เองเป็น Template หรือ Meta-Model สำหรับใช้สร้าง **FOM** และ **SOM**

ในทางปฏิบัติ OMT ถูกเขียนเป็นไฟล์ XML ตาม XML Schema ของ IEEE (namespace **http://standards.ieee.org/IEEE1516-2010**) ซึ่งเรียกว่า DIF (Data Interchange Format) เครื่องมือแต่ละอย่างจึงอ่านไฟล์ของกันและกันได้เพราะเป็น Format เดียวกัน


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model





© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

```
<objects>
    <objectClass>
        <name>HLAobjectRoot</name>
        <objectClass>
            <name>Food</name>
            <objectClass>
                <name>Drink</name>
                <objectClass>
                    <name>Water</name>
                    <sharing>PublishSubscribe</sharing>
                    <semantics>Tap water</semantics>
                </objectClass>
                <objectClass>
                    <name>Coffee</name>
                    <sharing>PublishSubscribe</sharing>
                    <semantics>Brewed coffee</semantics>
                </objectClass>
                <objectClass>
                    <name>Soda</name>
                    <sharing>PublishSubscribe</sharing>
                    <semantics>Carbonated beverage</semantics>
                    <attribute>
                        <name>Flavor</name>
                        <dataType>FlavorType</dataType>
                        <updateType>Conditional</updateType>
                        <updateCondition>Customer request</updateCondition>
                        <ownership>NoTransfer</ownership>
                        <sharing>PublishSubscribe</sharing>
```

XML File




© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ความหมายของ FOM SOM และ OMT
- Object, Attribute, Interaction และ Parameter
- โครงสร้าง FOM และชนิดข้อมูล
- Attribute และ Agreement ข้อมูล
- วิธีเขียน FOM/SOM - เริ่มต้นเขียน FOM ตามโจทย์ที่จะเรียน




© Zyntelligent Co., Ltd.

# Object, Attribute, Interaction และ Parameter

ใน HLA ข้อมูลที่ Federate แลกเปลี่ยนกันมี 2 แบบเท่านั้น

| ข้อมูล | คำอธิบาย | ตัวอย่าง |
|---|---|---|
| Object + Attribute | ข้อมูลที่มีอยู่ต่อเนื่อง และ มีสถานะเปลี่ยนไปตามเวลา | รถถัง เครื่องบิน จานอาหาร |
| Interaction + Parameter | เหตุการณ์ที่เกิดขึ้น (single-time event) | การยิง การชน การเสิร์ฟอาหาร |


© Zyntelligent Co., Ltd.

# Object, Attribute, Interaction และ Parameter

**Object class** คือ ประเภท ของสิ่งที่นิยามไว้ใน FOM คล้ายกับ class ใน Java และจัดเป็นลำดับชั้นที่ขึ้นต้นด้วย HLAobjectRoot เสมอ เช่น HLAobjectRoot.Rtaf.Aircraft

**Object instance** คือ ตัว object จริงที่ federate สร้างขึ้นตอนrun-time คล้ายกับการ new Aircraft() ใน Java

- RTI ระบุ instance ด้วย ObjectInstanceHandle
- แต่ละ instance มีชื่อที่ไม่ซ้ำกันทั้ง federation เช่น HLAobject_42 หรือชื่อที่เราทำ reservation เอง
- instance จะคงอยู่จนกว่าจะถูกลบหรือ federate ที่เป็นเจ้าของได้ resign ออกจาก federation

**Attribute** คือ สถานะ ของ object เช่น Flavor ของ Food หรือ Velocity ของเครื่องบิน คุณสมบัติที่สำคัญคือ

- Attribute สามารถ update ทีละบางตัวได้โดยไม่ต้องส่งทั้งหมด
- Federate เลือก subscribe เฉพาะ attribute ที่สนใจได้
- Attribute หนึ่งของ instance หนึ่ง มีเจ้าของ (federate) ได้คนเดียว และมีแต่เจ้าของเท่านั้นที่ update ได้
- ข้อมูลจะเป็น byte[] โดยต้อง encode และ decode เองตาม datatype ใน FOM


© Zyntelligent Co., Ltd.

# Object, Attribute, Interaction และ Parameter

**Interaction** คือ เหตุการณ์ ที่ส่งแล้วจบในครั้งเดียว

- ไม่มี instance และไม่มี handle ของ instance
- ไม่มีเจ้าของ ใครที่ publish ไว้ก็ส่งได้
- ไม่มีสถานะค้างอยู่ ถ้า federate ไม่ได้ subscribe ตอนที่ส่ง ก็จะพลาดเหตุการณ์นั้นไปเลย
- ลำดับชั้นขึ้นต้นด้วย HLAinteractionRoot เสมอ

**Parameter** คือ ข้อมูลประกอบ ของ interaction เช่น interaction WeaponFire มี parameter FiringObject, TargetLocation และ MunitionType

**ความต่างของ Parameter กับ Attribute ของ Object**

- parameter ส่งไปทั้งชุดพร้อมกัน ใน interaction เดียว
- subscribe interaction class แล้วจะได้ parameter ทั้งหมด จะเลือก subscribe ระดับ parameter ไม่ได้
- ไม่มีเจ้าของ (ownership)


© Zyntelligent Co., Ltd.

# Object, Attribute, Interaction และ Parameter

**การเปรียบเทียบ Object กับ Interaction**

| หัวข้อ | Object / Attribute | Interaction / Parameter |
|---|---|---|
| **ลักษณะ** | สถานะที่คงอยู่ | เหตุการณ์ (Single-time event) |
| **ราก hierarchy** | HLAobjectRoot | HLAinteractionRoot |
| **มี instance** | มี (ObjectInstanceHandle) | ไม่มี |
| **ความเป็นเจ้าของ** | มี และโอนได้ | ไม่มี |
| **การส่ง** | updateAttributeValues | sendInteraction |
| **การรับ** | reflectAttributeValues | receiveInteraction |
| **ส่งบางส่วน** | ได้ ส่งเฉพาะ attribute ที่เปลี่ยน | ไม่ได้ ต้องส่งทั้งชุด |
| **Subscribe ระดับ** | Attribute | Class ทั้งหมด |
| **Federate ที่มาทีหลัง** | ขอค่าล่าสุดได้ | พลาดเหตุการณ์เก่า |
| **Lifecycle callback** | discover / remove | ไม่มี |


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ความหมายของ FOM SOM และ OMT
- Object, Attribute, Interaction และ Parameter
- โครงสร้าง FOM และชนิดข้อมูล
- Attribute และ Agreement ข้อมูล
- วิธีเขียน FOM/SOM - เริ่มต้นเขียน FOM ตามโจทย์ที่จะเรียน




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

```
<?xml version="1.0" encoding="UTF-8"?>
<objectModel xmlns="http://standards.ieee.org/IEEE1516-2010" xmlns:xsi=".....">

    <modelIdentification> ... </modelIdentification>   <!-- 1  -->
    <serviceUtilization>  ... </serviceUtilization>    <!-- 2  -->
    <objects>             ... </objects>               <!-- 3  -->
    <interactions>        ... </interactions>          <!-- 4  -->
    <dimensions>          ... </dimensions>            <!-- 5  -->
    <time>                ... </time>                  <!-- 6  -->
    <tags>                ... </tags>                  <!-- 7  -->
    <synchronizations>    ... </synchronizations>      <!-- 8  -->
    <transportations>     ... </transportations>       <!-- 9  -->
    <switches>            ... </switches>              <!-- 10 -->
    <updateRates>         ... </updateRates>           <!-- 11 -->
    <dataTypes>           ... </dataTypes>             <!-- 12 -->
    <notes>               ... </notes>                 <!-- 13 -->


</objectModel>
```


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

| หัวข้อ | คำอธิบาย |
|---|---|
| Model Identification | ข้อมูลประจำตัว |
| serviceUtilization | RTI service ที่ใช้ |
| Objects | Object Class + Attribute |
| Interactions | Interaction Class + Parameter |
| Dimensions | DDM Dimensions |
| Time | ชนิดของ timestamp / lookahead |
| Tags | ชนิดของ user-supplied tag |
| Synchronizations | sync point |
| Transportations | reliable / best effort |
| Switches | ค่าเริ่มต้นของ RTI |
| UpdateRates | อัตรา update |
| DataTypes | นิยามชนิดข้อมูล |
| Notes | หมายเหตุ |




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

1

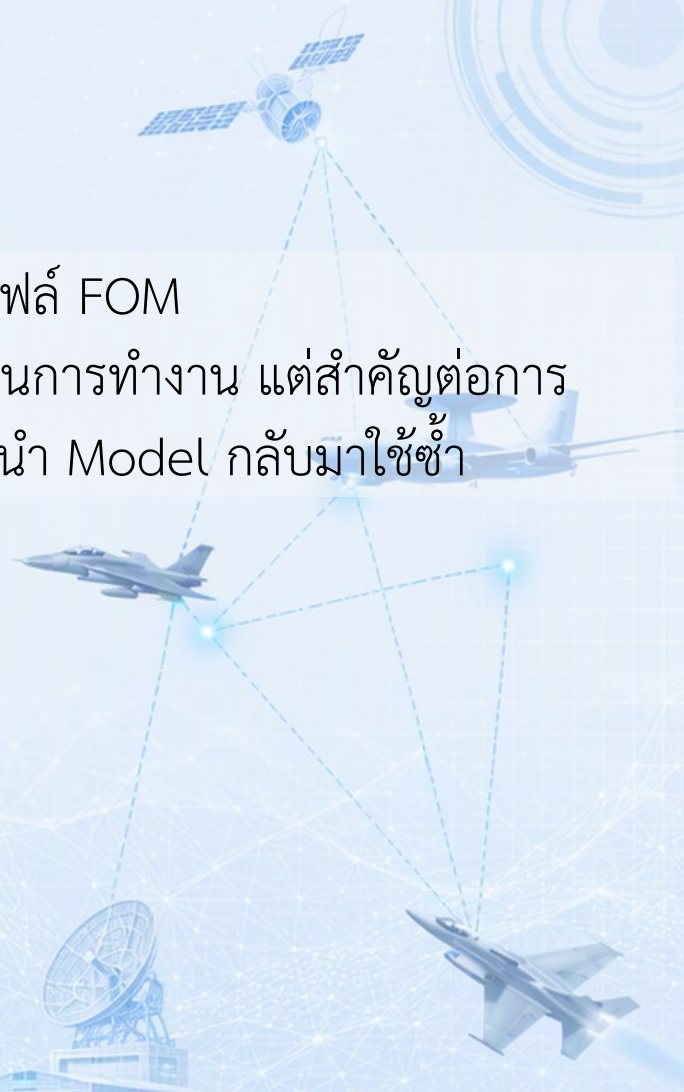
```
<modelIdentification>
    <name>VehicleFOM</name>
    <type>FOM</type>          <!-- FOM หรือ SOM -->
    <version>1.0</version>

    <modificationDate>2025-01-15</modificationDate>
    <securityClassification>Unclassified</securityClassification>
    <purpose>Aircraft Sim</purpose>
    <applicationDomain>Training</applicationDomain>
    <description>FOM module for Aircraft Sim</description>

    <poc>
        <pocType>Primary author</pocType>
        <pocName>Somchai</pocName>
        <pocOrg>My Lab</pocOrg>
        <pocEmail>somchai@example.com</pocEmail>
    </poc>

</modelIdentification>
```

เป็นข้อมูลเบื้องต้นของไฟล์ FOM
โดยส่วนนี้ RTI ไม่ได้ใช้ในการทำงาน แต่สำคัญต่อการจัดการเวอร์ชันและการนำ Model กลับมาใช้ซ้ำ




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

2

```
<serviceUtilization>
    <connect isUsed="true"/>
    <updateAttributeValues isUsed="true"/>
    <timeAdvanceRequest isUsed="true"/>
</serviceUtilization>
```

- ใช้ประกาศว่าโมเดลเรียก service ใดบ้าง
- มักใช้ใน SOM มากกว่า FOM


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

3

```
<objects>
    <objectClass>
        <name>HLAobjectRoot</name>
        <objectClass>
            <name>Vehicle</name>
            <sharing>PublishSubscribe</sharing>
            <semantics>ยานพาหนะภาคพื้นดิน</semantics>
            <attribute>
                <name>Position</name>
                <dataType>Vector3</dataType>
                <updateType>Periodic</updateType>
                <updateCondition>VehicleRate</updateCondition>
                <ownership>DivestAcquire</ownership>
                <sharing>PublishSubscribe</sharing>
                <transportation>HLAbestEffort</transportation>
                <order>Receive</order>
                <semantics>ตำแหน่งในพิกัดโลก (เมตร)</semantics>
            </attribute>
        </objectClass>
    </objectClass>
</objects>
```

เป็นข้อมูล Object Class และ Attribute ที่ใช้ภายใน Federation/Simulator

| Element | ค่าที่ใช้ได้ | ความหมาย |
|---|---|---|
| dataType | ชื่อ datatype ที่นิยามไว้ | รูปแบบการ encode |
| updateType | Static / Periodic / Conditional / NA | ไม่เปลี่ยน / ส่งเป็นรอบ / ส่งเมื่อมีเงื่อนไข |
| updateCondition | ข้อความ หรือชื่อ updateRate | เงื่อนไขในการส่ง เช่น "On change" |
| ownership | Divest / Acquire / DivestAcquire / NoTransfer | ปล่อยหรือรับ ownership ได้หรือไม่ |
| sharing | Publish / Subscribe / PublishSubscribe / Neither | บทบาทในการแลกเปลี่ยน |
| dimensions | รายชื่อ dimension | ใช้กรองด้วย DDM |
| transportation | HLAreliable / HLAbestEffort | วิธีส่ง |
| order | TimeStamp / Receive | TSO หรือ RO |


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

4

```
<interactions>
    <interactionClass>
        <name>HLAinteractionRoot</name>
        <interactionClass>
            <name>Collision</name>
            <sharing>PublishSubscribe</sharing>
            <transportation>HLAreliable</transportation>
            <order>TimeStamp</order>
            <semantics>ยานพาหนะชนกัน</semantics>
            <parameter>
                <name>Location</name>
                <dataType>Vector3</dataType>
                <semantics>จุดที่ชน</semantics>
            </parameter>
            <parameter>
                <name>Severity</name>
                <dataType>HLAfloat64BE</dataType>
            </parameter>
        </interactionClass>
    </interactionClass>
</interactions>
```

เป็นข้อมูล Interaction ภายใน Federation / Simulator




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

5

```
<dimensions>
    <dimension>
        <name>X_Zone</name>
        <dataType>HLAinteger32BE</dataType>
        <upperBound>1000</upperBound>
        <normalization>linear(X, 0, 10000)</normalization>
        <value>[0..1000)</value>      <!-- ค่าเริ่มต้น หรือ Excluded -->
    </dimension>
</dimensions>
```

DDM Dimentions

6

```
<time>
    <timeStamp>
        <dataType>HLAfloat64Time</dataType>
        <semantics>วินาทีนับจากเริ่มจำลอง</semantics>
    </timeStamp>
    <lookahead>
        <dataType>HLAfloat64Time</dataType>
    </lookahead>
</time>
```

**ชนิดข้อมูลของเวลา**

ต้องสอดคล้องกับ time factory ที่ใช้ในโค้ด เช่น HLAfloat64TimeFactory
หรือ HLAinteger64TimeFactory


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

7

```
<tags>
    <updateReflectTag>
        <dataType>HLAunicodeString</dataType>
        <semantics>trace ID สำหรับ debug</semantics>
    </updateReflectTag>
</tags>
```

**ชนิดข้อมูลของ User-supplied tag**

กำหนดได้แยกตาม service ได้แก่ updateReflectTag, sendReceiveTag, deleteRemoveTag, divestitureRequestTag, divestitureCompletionTag, acquisitionRequestTag และ requestUpdateTag

8

```
<synchronizations>
    <synchronizationPoint>
        <label>ReadyToRun</label>
        <dataType>NA</dataType>                    <!-- ชนิดของ tag ที่แนบมา -->
        <capability>RegisterAchieve</capability> <!-- Register / Achieve / RegisterAchieve / NoSynch -->
        <semantics>ทุก federate พร้อมเริ่มจำลอง</semantics>
    </synchronizationPoint>
</synchronizations>
```

**ข้อมูล Sync Point**


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

9

```
<transportations>
    <transportation>
        <name>LowLatency</name>
        <reliable>No</reliable>
        <semantics>ช่องทางเฉพาะของ RTI</semantics>
    </transportation>
</transportations>
```

**ข้อมูล transportations**

HLAreliable และ HLAbestEffort นิยามไว้ใน MIM แล้ว ถ้า RTI รองรับแบบอื่นเพิ่ม ให้นิยามที่นี่

10

```
<switches>
    <autoProvide isEnabled="true"/>
    <conveyRegionDesignatorSets isEnabled="false"/>
    <conveyProducingFederate isEnabled="false"/>
    <attributeScopeAdvisory isEnabled="false"/>
    <attributeRelevanceAdvisory isEnabled="false"/>
    <objectClassRelevanceAdvisory isEnabled="false"/>
    <interactionRelevanceAdvisory isEnabled="false"/>
    <serviceReporting isEnabled="false"/>
    <exceptionReporting isEnabled="false"/>
    <delaySubscriptionEvaluation isEnabled="false"/>
    <automaticResignAction resignAction="CancelThenDeleteThenDivest"/>
</switches>
```

ข้อมูลค่าเริ่มต้นของ RTI


© Zyntelligent Co., Ltd.

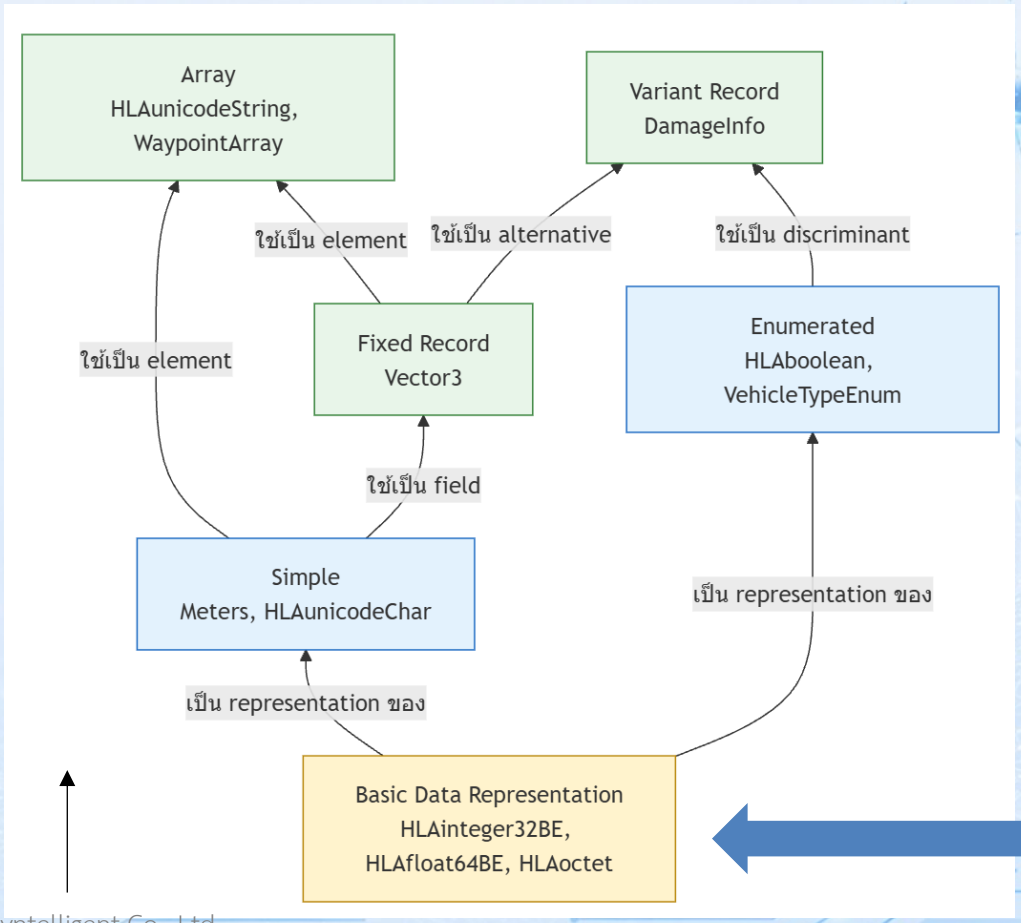
# โครงสร้าง FOM และชนิดข้อมูล



```
<dataTypes>
    <basicDataRepresentations/>
    <simpleDataTypes>
        <simpleData>
            <name>Meters</name>
            <representation>HLAfloat64BE</representation>
            <units>m</units>
            <resolution>0.001</resolution>
            <accuracy>NA</accuracy>
        </simpleData>
    </simpleDataTypes>
    <enumeratedDataTypes>
        <enumeratedData>
            <name>VehicleTypeEnum</name>
            <representation>HLAinteger32BE</representation>
            <enumerator><name>Car</name><value>0</value></enumerator>
            <enumerator><name>Truck</name><value>1</value></enumerator>
            <enumerator><name>Tank</name><value>2</value></enumerator>
        </enumeratedData>
        <enumeratedData>
            <name>DamageKindEnum</name>
            <representation>HLAinteger32BE</representation>
            <enumerator><name>None</name><value>0</value></enumerator>
            <enumerator><name>Mobility</name><value>1</value></enumerator>
            <enumerator><name>Destroyed</name><value>2</value></enumerator>
        </enumeratedData>
    </enumeratedDataTypes>
    <arrayDataTypes>
        <arrayData>
            <name>WaypointArray</name>
            <dataType>Vector3</dataType>
            <cardinality>Dynamic</cardinality>
            <encoding>HLAvariableArray</encoding>
        </arrayData>
    </arrayDataTypes>
    <fixedRecordDataTypes>
        <fixedRecordData>
            <name>Vector3</name>
            <encoding>HLAfixedRecord</encoding>
            <field><name>X</name><dataType>Meters</dataType></field>
            <field><name>Y</name><dataType>Meters</dataType></field>
            <field><name>Z</name><dataType>Meters</dataType></field>
        </fixedRecordData>
    </fixedRecordDataTypes>
    <variantRecordDataTypes>
        <variantRecordData>
```

## ข้อมูล Data Type

เป็นส่วนที่ใช้ Define ข้อมูลที่จะใช้ใน Federation





© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

12

```
<notes>
  <note>
    <label>Note1</label>
    <semantics>text</semantics>
  </note>
</notes>
```

**ข้อมูล Note**

หมายเหตุ




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type แต่ละแบบ

| หมวด | ประเภท | สร้างจาก | ใช้ทำอะไร |
|---|---|---|---|
| Basic | ระดับต่ำสุด | กำหนดขนาด bit และ endian โดยตรง | การจัดเรียง byte บนสาย |
| Simple | Scalar ที่มีความหมาย | Basic | ใส่หน่วย ความละเอียด ความแม่นยำ |
| Enumerated | ค่าจากรายการ | Basic (ตัวเลข) | สถานะหรือประเภท |
| Array | Composite | type ใดก็ได้ | ลิสต์ สตริง |
| Fixed Record | Composite | type ใดก็ได้ | struct |
| Variant Record | Composite | Enumerated + type ใดก็ได้ | union ที่มีตัวบอกชนิด |


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type

### 11.1) Basic Data Representation

```
<basicData>
    <name>HLAinteger32BE</name>
    <size>32</size>
    <interpretation>Integer in the range [-2^31, 2^31 - 1]</interpretation>
    <endian>Big</endian>
    <encoding>32-bit two's complement signed integer</encoding>
</basicData>
```

### Basic type ที่มีให้

| ชื่อ | ขนาด | ความหมาย |
|---|---|---|
| HLAoctet | 8 bit | 1 byte |
| HLAoctetPairBE / LE | 16 bit | 2 byte |
| HLAinteger16BE / LE | 16 bit | short |
| HLAinteger32BE / LE | 32 bit | int |
| HLAinteger64BE / LE | 64 bit | long |
| HLAfloat32BE / LE | 32 bit | float (IEEE 754) |
| HLAfloat64BE / LE | 64 bit | double (IEEE 754) |

**BE = Big Endian, LE = Little Endian** ควรใช้ **BE** เป็นมาตรฐาน ทั้ง federation


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type

### 11.2) Simple Data Types

คือ basic type ที่เพิ่มความหมายทางโดเมนเข้าไป

```
<simpleData>
    <name>Meters</name>
    <representation>HLAfloat64BE</representation>
    <units>m</units>
    <resolution>0.001</resolution>
    <accuracy>±0.01</accuracy>
    <semantics>ระยะทางเป็นเมตร</semantics>
</simpleData>
```

**Simple Data Types ที่มีให้**

- HLAASCIIchar (HLAoctet)
- HLAunicodeChar (HLAoctetPairBE)
- HLAbyte (HLAoctet)


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type

### 11.3) Enumerated Data Types

```
<enumeratedData>
    <name>VehicleTypeEnum</name>
    <representation>HLAinteger32BE</representation>
    <semantics>ประเภทยานพาหนะ</semantics>
    <enumerator><name>Car</name><value>0</value></enumerator>
    <enumerator><name>Truck</name><value>1</value></enumerator>
    <enumerator><name>Tank</name><value>2</value></enumerator>
</enumeratedData>
```




© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type

### 11.4) Array Data Types

```
<arrayData>
    <name>WaypointArray</name>
    <dataType>Vector3</dataType>
    <cardinality>Dynamic</cardinality>
    <encoding>HLAvariableArray</encoding>
    <semantics>เส้นทางที่วางแผนไว้</semantics>
</arrayData>
```

| Encoding | Cardinality | รูปแบบ |
|---|---|---|
| HLAfixedArray | ตัวเลขคงที่ เช่น 3 | มีแค่ element เรียงต่อกัน |
| HLAvariableArray | Dynamic | HLAinteger32BE จำนวน element แล้วตามด้วย element |

### Array Data Types ที่มีให้

| ชื่อ | Element | ใช้กับ |
|---|---|---|
| HLAASCIIstring | HLAASCIIchar | ข้อความ ASCII |
| HLAunicodeString | HLAunicodeChar | ข้อความ Unicode (UTF-16BE) |
| HLAopaqueData | HLAbyte | ข้อมูลดิบที่ encode เอง |


© Zyntelligent Co., Ltd.

# โครงสร้าง FOM และชนิดข้อมูล

## Data Type

### 11.5) Fixed Record Data Types

เทียบได้กับ struct ใน C หรือ record ใน Java

```
<fixedRecordData>
    <name>Vector3</name>
    <encoding>HLAfixedRecord</encoding>
    <semantics>เวกเตอร์ 3 มิติ</semantics>
    <field><name>X</name><dataType>Meters</dataType></field>
    <field><name>Y</name><dataType>Meters</dataType></field>
    <field><name>Z</name><dataType>Meters</dataType></field>
</fixedRecordData>
```

### 11.6) Variant Record Data Types

```
<enumeratedData>
    <name>DamageKindEnum</name>
    ....
</enumeratedData>

<variantRecordData>
    <name>DamageInfo</name>
    <discriminant>Kind</discriminant>
    <dataType>DamageKindEnum</dataType>
    <alternative>
        <enumerator>Mobility</enumerator>
        <name>SpeedReduction</name>
        <dataType>HLAfloat64BE</dataType>
    </alternative>
    <alternative>
        <enumerator>Destroyed</enumerator>
        <name>DestroyedAt</name>
        <dataType>Vector3</dataType>
    </alternative>
    <encoding>HLAvariantRecord</encoding>
</variantRecordData>
```

เทียบได้กับ union ที่มีตัวบอกชนิด (tagged union) โดยค่า enum ใน discriminant จะเป็นตัวกำหนดว่า field ไหนถูกส่ง


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ความหมายของ FOM SOM และ OMT
- Object, Attribute, Interaction และ Parameter
- โครงสร้าง FOM และชนิดข้อมูล
- Attribute และ Agreement ข้อมูล
- **วิธีเขียน FOM/SOM - เริ่มต้นเขียน FOM ตามโจทย์ที่จะเรียน**




© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

## การจำลองการบินของเครื่องบินรบ (FighterFOM.xml)

**Federate ที่มีใน Federation**

- เครื่องบินรบ
- เรดาร์ภาคพื้นดิน

**ข้อมูลที่ต้องใช้เบื้องต้น**

**Object**: ชื่อ ตำแหน่ง ความเร็ว ทิศทาง และสถานะ

**Interaction**: RadarContact (เรดาร์ตรวจพบเป้า) และ WeaponFire (การยิงอาวุธ)


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

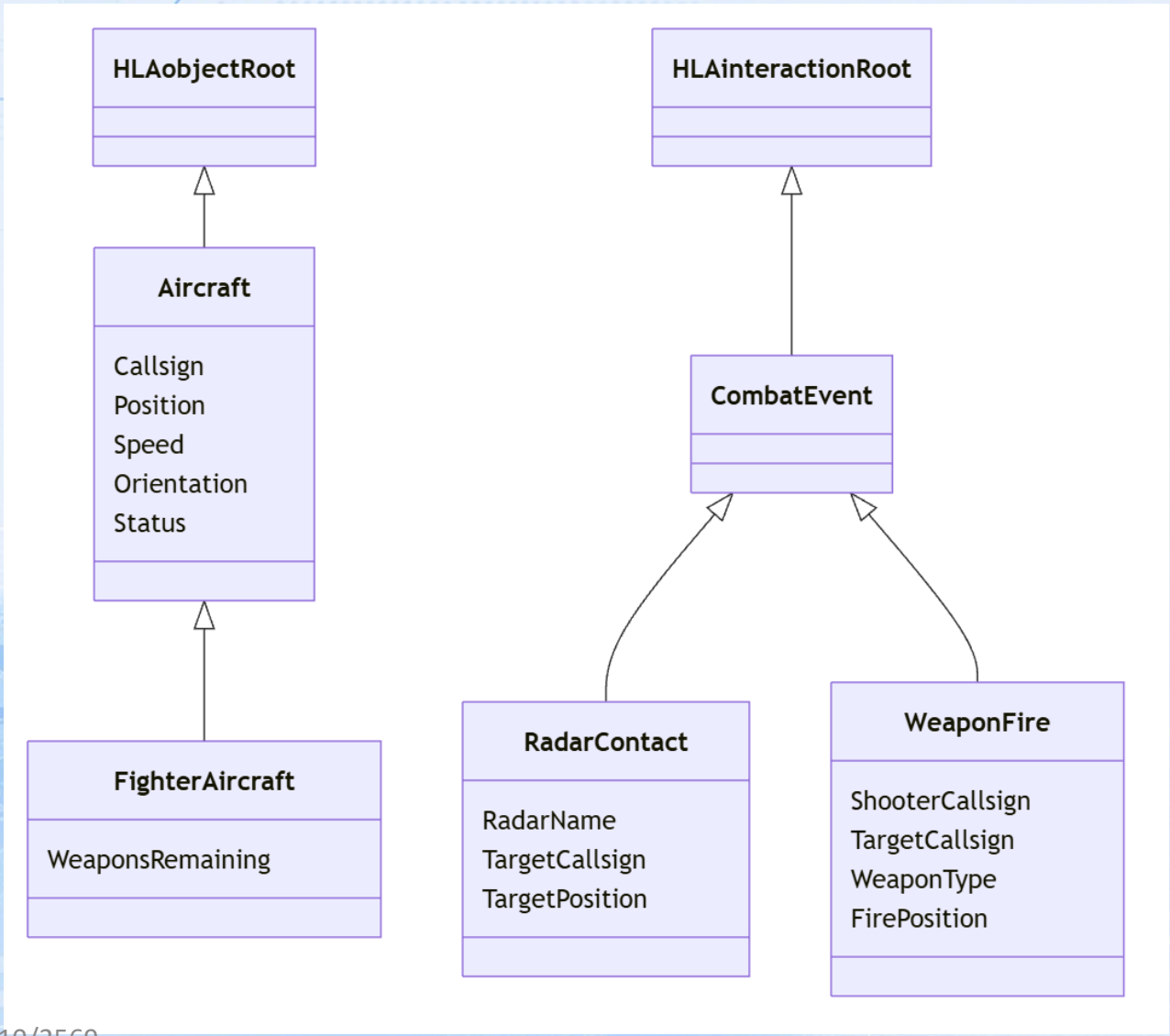
## การแลกเปลี่ยนข้อมูล

| ข้อมูล | FighterSim | RadarSim |
|---|---|---|
| ชื่อ ตำแหน่ง ความเร็ว ทิศทาง สถานะของเครื่องบิน | P / S | S |
| จำนวนอาวุธคงเหลือ | P / S | – |
| เรดาร์ตรวจพบเป้า (RadarContact) | S | P |
| การยิงอาวุธ (WeaponFire) | P / S | – |

P = Publish, S = Subscrib


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM




**Aircraft** เก็บ attribute ที่เครื่องบินทุกประเภทมีร่วมกัน

**FighterAircraft** เพิ่มเฉพาะข้อมูลของเครื่องบินรบ

**RadarSim** subscribe เฉพาะ Aircraft

HLAobjectRoot.Aircraft

HLAobjectRoot.Aircraft.FighterAircraft

HLAinteractionRoot.CombatEvent.RadarContact

HLAinteractionRoot.CombatEvent.WeaponFire


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

## Object / Attribute

| Class | Attribute | ความหมาย | Datatype | updateType | ownership | order |
|---|---|---|---|---|---|---|
| Aircraft | Callsign | ชื่อเรียกขาน เช่น Viper01 | HLAunicodeString | Static | NoTransfer | Receive |
| Aircraft | Position | ละติจูด ลองจิจูด และความสูง | GeodeticPosition | Periodic | DivestAcquire | TimeStamp |
| Aircraft | Speed | ความเร็ว | MetersPerSecond | Periodic | DivestAcquire | TimeStamp |
| Aircraft | Orientation | heading, pitch และ roll | Orientation | Periodic | DivestAcquire | TimeStamp |
| Aircraft | Status | สถานะของเครื่องบิน | AircraftStatusEnum | Conditional | DivestAcquire | TimeStamp |
| FighterAircraft | WeaponsRemaining | จำนวนอาวุธคงเหลือ | HLAinteger32BE | Conditional | DivestAcquire | TimeStamp |


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

Interaction

| Interaction | Parameter | Datatype | ความหมาย |
|---|---|---|---|
| RadarContact | RadarName | HLAunicodeString | ชื่อเรดาร์ที่ตรวจพบ |
| RadarContact | TargetCallsign | HLAunicodeString | ชื่อเรียกขานของเป้า |
| RadarContact | TargetPosition | GeodeticPosition | ตำแหน่งที่ตรวจพบ |
| WeaponFire | ShooterCallsign | HLAunicodeString | ผู้ยิง |
| WeaponFire | TargetCallsign | HLAunicodeString | เป้าหมาย |
| WeaponFire | WeaponType | WeaponTypeEnum | ประเภทอาวุธ |
| WeaponFire | FirePosition | GeodeticPosition | ตำแหน่งที่ยิง |


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

## Datatypes

| ชื่อ | หมวด | นิยาม |
|---|---|---|
| Meters | Simple | HLAfloat64BE, หน่วย m |
| MetersPerSecond | Simple | HLAfloat64BE, หน่วย m/s |
| AngleDegrees | Simple | HLAfloat64BE, หน่วย องศา |
| AircraftStatusEnum | Enumerated | Parked=0, Airborne=1, Damaged=2, Destroyed=3 |
| WeaponTypeEnum | Enumerated | Gun=0, ShortRangeMissile=1, MediumRangeMissile=2 |
| GeodeticPosition | Fixed Record | Latitude, Longitude (AngleDegrees), Altitude (Meters) |
| Orientation | Fixed Record | Heading, Pitch, Roll (AngleDegrees) |


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

## Semantics

| ค่า | ข้อตกลง |
|---|---|
| Latitude | –90 ถึง +90 องศา ค่าบวกคือซีกโลกเหนือ (WGS84) |
| Longitude | –180 ถึง +180 องศา ค่าบวกคือซีกโลกตะวันออก |
| Altitude | เมตรเหนือระดับน้ำทะเลปานกลาง (MSL) |
| Heading | 0–360 องศา 0 คือทิศเหนือจริง หมุนตามเข็มนาฬิกา |
| Pitch | –90 ถึง +90 องศา ค่าบวกคือเชิดหัวขึ้น |
| Roll | –180 ถึง +180 องศา ค่าบวกคือเอียงปีกขวาลง |


© Zyntelligent Co., Ltd.

# วิธีเขียน FOM/SOM

FighterFOM.xml

```xml
<?xml version="1.0" encoding="UTF-8"?>
<objectModel xmlns="http://standards.ieee.org/IEEE1516-2010"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://standards.ieee.org/IEEE1516-2010
                 http://standards.ieee.org/downloads/1516/1516.2-2010/IEEE1516-DIF-2010.xsd">

    <!-- ==================== 1. Identification ==================== -->
    <modelIdentification>
        <name>FighterFOM</name>
        <type>FOM</type>
        <version>1.0</version>
        <modificationDate>2025-01-15</modificationDate>
        <securityClassification>Unclassified</securityClassification>
        <description>FOM สำหรับจำลองเครื่องบินรบ เรดาร์ และเหตุการณ์การรบ</description>
    </modelIdentification>

    <!-- ==================== 2. Objects ==================== -->
    <objects>
        <objectClass>
            <name>HLAobjectRoot</name>

            <objectClass>
                <name>Aircraft</name>
                <sharing>PublishSubscribe</sharing>
                <semantics>อากาศยานทุกประเภท</semantics>

                <attribute>
                    <name>Callsign</name>
                    <dataType>HLAunicodeString</dataType>
                    <updateType>Static</updateType>
                    <updateCondition>NA</updateCondition>
                    <ownership>NoTransfer</ownership>
                    <sharing>PublishSubscribe</sharing>
                    <transportation>HLAreliable</transportation>
                    <order>Receive</order>
                    <semantics>ชื่อเรียกขานที่ไม่ซ้ำกัน เช่น Viper01</semantics>
                </attribute>

                <attribute>
                    <name>Position</name>
                    <dataType>GeodeticPosition</dataType>
                    <updateType>Periodic</updateType>
                    <updateCondition>AircraftRate</updateCondition>
                    <ownership>DivestAcquire</ownership>
                    <sharing>PublishSubscribe</sharing>
                    <transportation>HLAreliable</transportation>
                    <order>TimeStamp</order>
                    <semantics>ตำแหน่งตามพิกัด WGS84</semantics>
                </attribute>
```


© Zyntelligent Co., Ltd.

# บทที่ 4
# เริ่มต้นสร้าง Federate / Federation




© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

| เครื่องมือ | วัตถุประสงค์ |
|---|---|
| Java JDK | พัฒนาและรันโปรแกรม Federate |
| ZyntRTI Library | Library สำหรับ HLA |
| IntelliJ IDEA / Eclipse | พัฒนาและ Debug โปรแกรม |
| Maven / Gradle | จัดการ Dependencies และ Build Project |
| FOM XML | กำหนดโครงสร้างข้อมูลที่ใช้แลกเปลี่ยนระหว่าง Federates |

- ติดตั้ง Java JDK และตรวจสอบด้วยคำสั่ง java -version
- ติดตั้ง ZyntRTI Library ตั้งค่า RTI_HOME และ Library Path
- ตรวจสอบว่าโปรแกรมสามารถเข้าถึง Library ได้
- เตรียมไฟล์ FOM และตรวจสอบ XML ให้ถูกต้องตามมาตรฐาน


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

**1. Environment Setup**
Java JDK + Zynt RTI

↓

**2. Project Initialization**
Dependencies + FOM

↓

**3. Federate Initialization**
RTIambassador + FederateAmbassador

↓

**4. Federation Management**
Connect → Create → Join

↓

**5. Federate Services**
Methods + Callback Handling

↓

**6. Synchronization**
Register → Achieve → Synchronized

↓

**7. Federation Cleanup**
Resign → Disconnect


© Zyntelligent Co., Ltd.
© Zyntelligent Co., Ltd.

89

# Federation Object Model / Simulation Object Model

- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- **เริ่มต้นตั้งค่า Project**
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

สร้าง Java Project และกำหนดโครงสร้างสำหรับพัฒนา Federate โดยแยกส่วนการเชื่อมต่อ RTI, การจัดการ Callback และการทำงานหลักของ Simulation

```
HLA-Federate/
├── pom.xml
├── lib/
│   └── portico.jar
├── src/main/
│   ├── java/com/rtaf/hla/
│   │   ├── Main.java
│   │   ├── AircraftFighterFederate.java
│   │   └── AircraftFighterFederateAmbassador.java
│   └── resources/
│       └── fom/
│           └── FighterFOM.xml
```


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

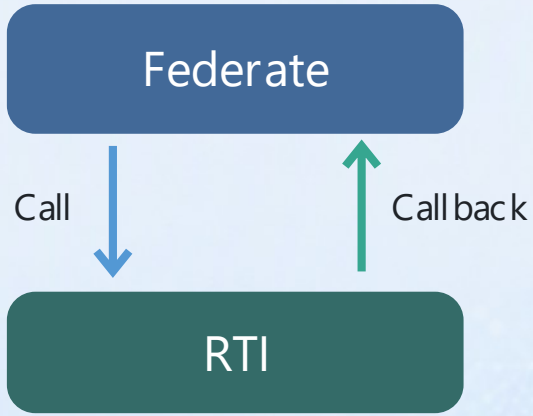
- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

**Federate** คือโปรแกรมหรือระบบจำลองที่เข้าร่วม Federation เพื่อแลกเปลี่ยนข้อมูลหรือทำงานร่วมกับ Federate อื่นผ่าน RTI การสร้าง Federate ต้องประกอบด้วยองค์ประกอบหลัก 2 ส่วน





© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- **การสร้าง Federation และ join Federation**
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

การสร้าง Federation และ join Federation

```
URL fomUrl = new File("src/main/resources/fom/FighterFOM.xml").toURI().toURL();

try {
    rtiAmbassador.createFederationExecution("RTAFFederation", fomUrl);
} catch (FederationExecutionAlreadyExists e) {
    System.out.println("Federation already exists");
}


rtiAmbassador.joinFederationExecution("federateName", "federateType", "federationName");
```


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- **การสร้าง Methods ที่จำเป็นต่อการใช้งาน**
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

| Method | หน้าที่ |
|---|---|
| initializeRTI() | สร้าง RTI Ambassador และ Federate Ambassador |
| connect() | เชื่อมต่อกับ RTI |
| createFederation() | สร้าง Federation Execution |
| joinFederation() | เข้าร่วม Federation |
| initializeHandles() | ดึง Object/Interaction Handles จาก FOM |
| publishAndSubscribe() | ประกาศข้อมูลที่ต้องการส่งและรับ |
| processCallbacks() | ประมวลผล Callback จาก RTI |
| runSimulation() | เริ่มต้น Simulation Loop |
| resignFederation() | ออกจาก Federation |
| disconnect() | ยุติการเชื่อมต่อ RTI |


© Zyntelligent Co., Ltd.
© Zyntelligent Co., Ltd.

97

# Federation Object Model / Simulation Object Model

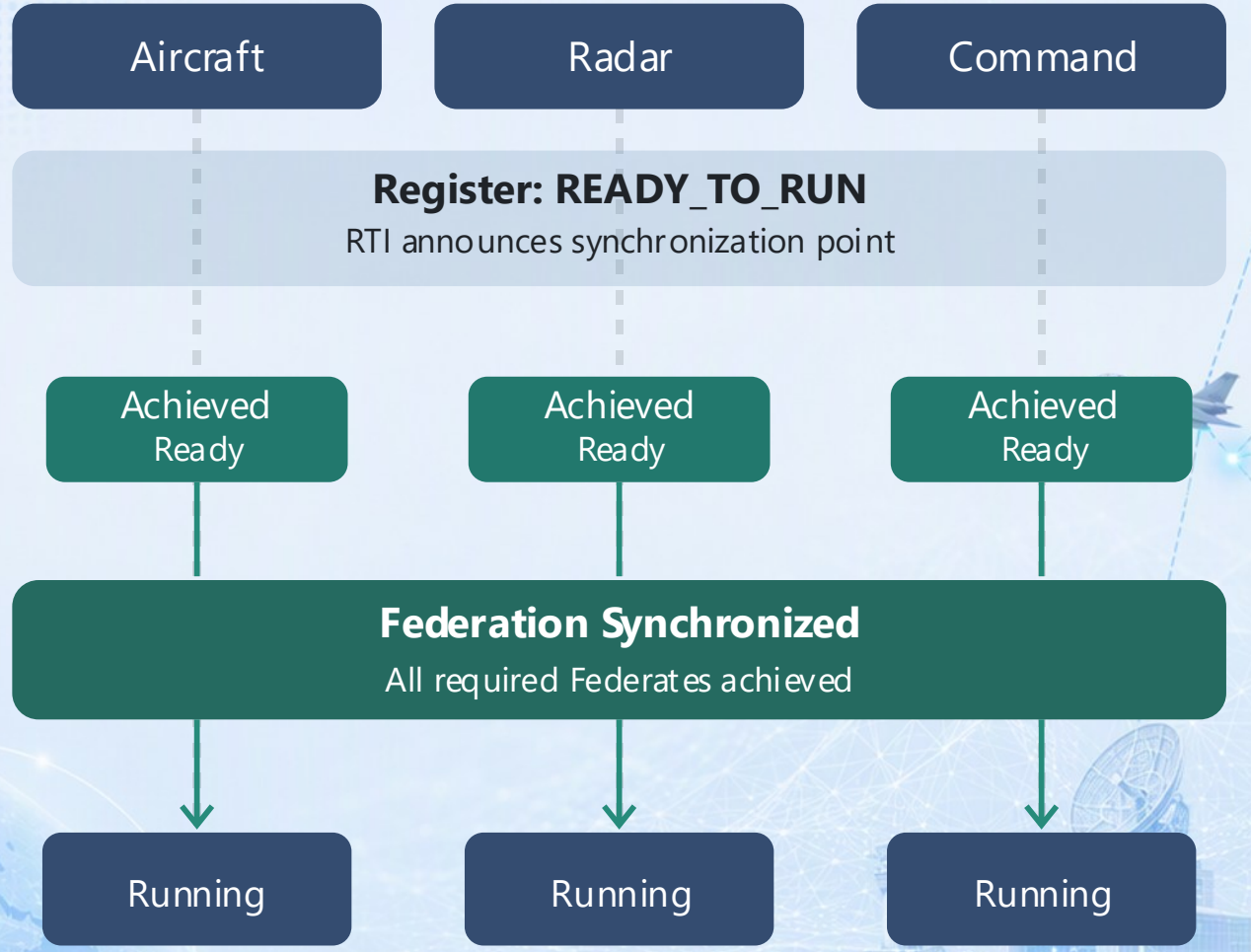
- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

Synchronization Point




Simulation starts together


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

**กระบวนการ Synchronization ประกอบด้วย**

- Federate ลงทะเบียน Synchronization Point กับ RTI
- RTI แจ้ง announceSynchronizationPoint() ให้สมาชิกที่เกี่ยวข้อง
- Federate แต่ละตัวตรวจสอบความพร้อมของตนเอง
- Federate เรียก synchronizationPointAchieved()
- RTI แจ้ง federationSynchronized() เมื่อสมาชิกที่อยู่ในขอบเขต Synchronization บรรลุเงื่อนไขครบ
- Federates ดำเนินการ Simulation ต่อได้


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

## Synchronization Point Methods & Callback

```
rtiAmbassador.registerFederationSynchronizationPoint(
    "READY_TO_RUN",
    null
);
```

```
rtiAmbassador.synchronizationPointAchieved(
    "READY_TO_RUN"
);
```

```
public volatile boolean isSynchronized = false;

@Override
public void federationSynchronized(
        String label,
        FederateHandleSet failedToSyncSet) {

    if ("READY_TO_RUN".equals(label)) {
        isSynchronized = true;
    }
}
```

```
while (!federateAmbassador.isSynchronized) {
    rtiAmbassador.evokeMultipleCallbacks(0.1, 0.5);
}
```


© Zyntelligent Co., Ltd.

# Federation Object Model / Simulation Object Model

- ตรวจสอบความพร้อม, ลงโปรแกรมที่จำเป็น
- เริ่มต้นตั้งค่า Project
- การสร้าง Federate
- การสร้าง Federation และ join Federation
- การสร้าง Methods ที่จำเป็นต่อการใช้งาน
- การใช้งาน Synchronization Point
- การ resign Federation และ disconnect




© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

เมื่อ Federate เสร็จสิ้นการทำงานหรือจำเป็นต้องออกจาก Federation ควรดำเนินการปิดการทำงานตามลำดับ เพื่อให้ RTI สามารถจัดการทรัพยากรได้อย่างถูกต้อง

**การ Resign Federation**

ใช้ resignFederationExecution() เพื่อยุติสถานะการเป็นสมาชิกของ Federation

**การ Destroy Federation**

กรณีที่ต้องการปิด Federation Execution หลังจากไม่มี Federate อื่นเข้าร่วมแล้ว

**การ Disconnect RTI**

หลังจาก Resign เรียบร้อยแล้ว จึงยุติการเชื่อมต่อ RTI


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

```
rtiAmbassador.resignFederationExecution(
    ResignAction.DELETE_OBJECTS
);

rtiAmbassador.destroyFederationExecution(
    "RTAFFederation"
);

rtiAmbassador.disconnect();
```


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation



**ลำดับการ Clean-up ที่แนะนำ**

1. Stop Simulation Loop
2. Delete Objects
3. Resign Federation
4. Destroy Federation (Optional)
5. Disconnect RTI
6. Terminate Application


© Zyntelligent Co., Ltd.

# เริ่มต้นสร้าง Federate / Federation

เมื่อ Federate เสร็จสิ้นการทำงานหรือจำเป็นต้องออกจาก Federation ควรดำเนินการปิดการทำงานตามลำดับ เพื่อให้ RTI สามารถจัดการทรัพยากรได้อย่างถูกต้อง

**การ Resign Federation**

ใช้ resignFederationExecution() เพื่อยุติสถานะการเป็นสมาชิกของ Federation

**การ Destroy Federation**

กรณีที่ต้องการปิด Federation Execution หลังจากไม่มี Federate อื่นเข้าร่วมแล้ว

**การ Disconnect RTI**

หลังจาก Resign เรียบร้อยแล้ว จึงยุติการเชื่อมต่อ RTI


© Zyntelligent Co., Ltd.

# บทที่ 5
# โครงสร้าง Federate Ambassador และ Callback Model


© Zyntelligent Co., Ltd.

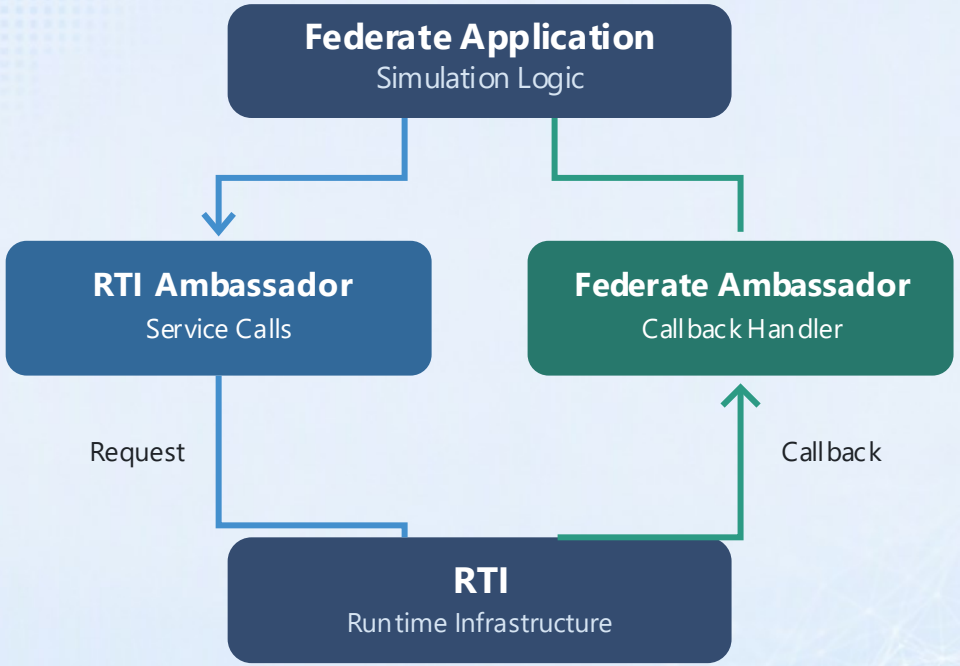
# โครงสร้าง Federate Ambassador และ Callback Model

- การสร้าง FederateAmbassador
- CallBack ต่างๆ ที่จำเป็นต่อการใช้งาน
- ทดสอบรัน Federate




© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model




Federate Ambassador เป็นส่วนประกอบสำคัญของ Federate ทำหน้าที่รับการแจ้งเตือนและข้อมูลจาก RTI ผ่าน Callback Methods

เมื่อเกิดเหตุการณ์ใน Federation เช่น การประกาศ Synchronization Point การสร้าง Object หรือการเปลี่ยนแปลง Attributes ระบบ RTI จะเรียก Callback ที่เกี่ยวข้องใน Federate Ambassador


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## โครงสร้างของ Federate Ambassador

สำหรับ IEEE 1516-2010 สามารถสร้าง Class โดยสืบทอดจาก NullFederateAmbassador ซึ่งมีการเตรียม Methods ว่างไว้แล้ว ทำให้ผู้พัฒนาสามารถ Override เฉพาะ Methods ที่ต้องการใช้งานได้

```
import hla.rti1516e.*;

public class FederateAmbassador
        extends NullFederateAmbassador {

    public volatile boolean isAnnounced = false;
    public volatile boolean isSynchronized = false;

    @Override
    public void announceSynchronizationPoint(
            String label, byte[] tag) {

        System.out.println("Announced: " + label);
        isAnnounced = true;
    }

    @Override
    public void federationSynchronized(
            String label,
            FederateHandleSet failedToSyncSet) {

        System.out.println("Synchronized: " + label);
        isSynchronized = true;
    }
}
```

# โครงสร้าง Federate Ambassador และ Callback Model

**การกำหนด Federate Ambassador ให้กับ RTI**

สร้าง Instance ของ Federate Ambassador และส่งให้ RTI เมื่อทำการเชื่อมต่อ

```
RTIambassador rtiAmbassador = RtiFactoryFactory.getRtiFactory().getRtiAmbassador();

FederateAmbassador fedAmb = new FederateAmbassador();

rtiAmbassador.connect(
    fedAmb,
    CallbackModel.HLA_EVOKED
);
```


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

- การสร้าง FederateAmbassador
- Callback ต่างๆ ที่จำเป็นต่อการใช้งาน
- ทดสอบรัน Federate




© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## การรับ Callback

| รูปแบบ | ลักษณะการทำงาน | ข้อควรพิจารณา |
|---|---|---|
| HLA_EVOKED | Federate เรียก Methods เพื่อประมวลผล Callback | ควบคุมลำดับการทำงานได้ง่าย |
| HLA_IMMEDIATE | RTI เรียก Callback ผ่าน Thread ของระบบ | ต้องจัดการ Thread Safety อย่างเหมาะสม |


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## Federation Management Callbacks

| Callback | หน้าที่ |
|---|---|
| synchronizationPointRegistrationSucceeded() | แจ้งว่าลงทะเบียน Synchronization Point สำเร็จ |
| synchronizationPointRegistrationFailed() | แจ้งว่าลงทะเบียนไม่สำเร็จ |
| announceSynchronizationPoint() | แจ้งว่ามี Synchronization Point |
| federationSynchronized() | แจ้งว่าการ Synchronization เสร็จสิ้นตามขอบเขตที่กำหนด |
| connectionLost() | แจ้งการสูญเสียการเชื่อมต่อกับ RTI (หาก RTI รองรับและตรวจพบ) |


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## Object Management Callbacks

ใช้สำหรับติดตาม Object Instances ภายใน Federation

| Callback | หน้าที่ |
|---|---|
| discoverObjectInstance() | ตรวจพบ Object Instance ใหม่ |
| reflectAttributeValues() | รับค่า Attributes ที่ถูก Update |
| removeObjectInstance() | แจ้งการลบ Object Instance |
| provideAttributeValueUpdate() | แจ้งให้เจ้าของ Attributes จัดเตรียมค่าอัปเดต |

Callback นี้มักเกิดขึ้นเมื่อ Federate สมัครรับ Object Class ที่เกี่ยวข้อง และมี Object Instance ที่ RTI ทำให้ Federate มองเห็นได้


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

**Interaction Management Callbacks**

ใช้สำหรับรับข้อมูล Interaction ที่ Federate อื่นส่งเข้ามา

Callback หลักคือ receiveInteraction()

```
@Override
public void receiveInteraction(
        InteractionClassHandle interactionClass,
        ParameterHandleValueMap parameters,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        SupplementalReceiveInfo info) {

    System.out.println("Interaction received");
}
```


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## Time Management Callbacks

| Callback | หน้าที่ |
|---|---|
| timeRegulationEnabled() | แจ้งว่าเปิด Time Regulation แล้ว |
| timeConstrainedEnabled() | แจ้งว่าเปิด Time Constrained แล้ว |
| timeAdvanceGrant() | แจ้งว่า RTI อนุญาตให้ก้าวหน้าไปยัง Logical Time ที่กำหนด |

สำหรับการเริ่มต้น สามารถใช้ Receive Order (RO) โดยยังไม่เปิด Time Management


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## Time Management Callbacks

| Callback | หน้าที่ |
|---|---|
| timeRegulationEnabled() | แจ้งว่าเปิด Time Regulation แล้ว |
| timeConstrainedEnabled() | แจ้งว่าเปิด Time Constrained แล้ว |
| timeAdvanceGrant() | แจ้งว่า RTI อนุญาตให้ก้าวหน้าไปยัง Logical Time ที่กำหนด |

สำหรับการเริ่มต้น สามารถใช้ Receive Order (RO) โดยยังไม่เปิด Time Management


© Zyntelligent Co., Ltd.
© Zyntelligent Co., Ltd.

118

# โครงสร้าง Federate Ambassador และ Callback Model

- การสร้าง FederateAmbassador
- Callback ต่างๆ ที่จำเป็นต่อการใช้งาน
- ทดสอบรัน Federate




© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

## ทดสอบรับ Callback ของ Synchronization Point

```java
import hla.rti1516e.*;

public class FederateAmbassador
        extends NullFederateAmbassador {

    public volatile boolean announced = false;
    public volatile boolean synchronizedFlag = false;

    @Override
    public void synchronizationPointRegistrationSucceeded(
            String label) {
        System.out.println("[CALLBACK] Registered: " + label);
    }

    @Override
    public void announceSynchronizationPoint(
            String label, byte[] tag) {
        System.out.println("[CALLBACK] Announced: " + label);
        if ("READY_TO_RUN".equals(label)) {
            announced = true;
        }
    }

    @Override
    public void federationSynchronized(
            String label,
            FederateHandleSet failedToSyncSet) {
        System.out.println("[CALLBACK] Synchronized: " + label);
        if ("READY_TO_RUN".equals(label)) {
            synchronizedFlag = true;
        }
    }
}
```

```java
public static void main(String[] args) throws Exception {
    RTIambassador rti = RtiFactoryFactory
            .getRtiFactory()
            .getRtiAmbassador();

    FederateAmbassador fedAmb =
            new FederateAmbassador();

    boolean connected = false;
    boolean joined = false;

    try {
        // 1. Connect
        rti.connect(fedAmb, CallbackModel.HLA_EVOKED);
        connected = true;
        System.out.println("[RTI] Connected");

        // 2. Create Federation
        URL fom = new File(
                "src/main/resources/fom/FighterFOM.xml"
        ).toURI().toURL();

        try {
            rti.createFederationExecution(
                    "RTAFFederation", fom);
            System.out.println("[RTI] Federation created");
        } catch (FederationExecutionAlreadyExists e) {
            System.out.println("[RTI] Federation exists");
        }

        // 3. Join Federation
        rti.joinFederationExecution(
                "TestFederate",
                "SimulationFederate",
                "TestFederation");
        joined = true;
        System.out.println("[RTI] Joined");

        // 4. Register Synchronization Point
        rti.registerFederationSynchronizationPoint(
                "READY_TO_RUN", null);
        .....

        .....

        .....
```


© Zyntelligent Co., Ltd.
© Zyntelligent Co., Ltd.

120

# โครงสร้าง Federate Ambassador และ Callback Model

Checklist

| รายการทดสอบ | ผลลัพธ์ที่ต้องการ |
|---|---|
| RTI Connection | เชื่อมต่อสำเร็จ |
| Federation Creation | สร้างหรือพบ Federation เดิม |
| Federate Join | เข้าร่วม Federation สำเร็จ |
| Callback Registration | ได้รับ Registration Succeeded |
| Callback Announcement | ได้รับ Announce Sync Point |
| Synchronization | ได้รับ Federation Synchronized |
| Federate Resign | ออกจาก Federation สำเร็จ |
| RTI Disconnect | ยุติการเชื่อมต่อสำเร็จ |


© Zyntelligent Co., Ltd.

# โครงสร้าง Federate Ambassador และ Callback Model

ตัวอย่าง Console Log

```
[RTI] Connected
[RTI] Federation created
[RTI] Joined
[CALLBACK] Registered: READY_TO_RUN
[CALLBACK] Announced: READY_TO_RUN
[CALLBACK] Synchronized: READY_TO_RUN
[SIM] Ready to run
[RTI] Resigned
[RTI] Disconnected
```

หากไม่ได้รับ Callback ให้ตรวจสอบว่าใช้ Callback Model ตรงตามที่กำหนดมีการเรียก **evokeMultipleCallbacks()** อย่างต่อเนื่อง และ Federate ยังอยู่ในสถานะ Joined



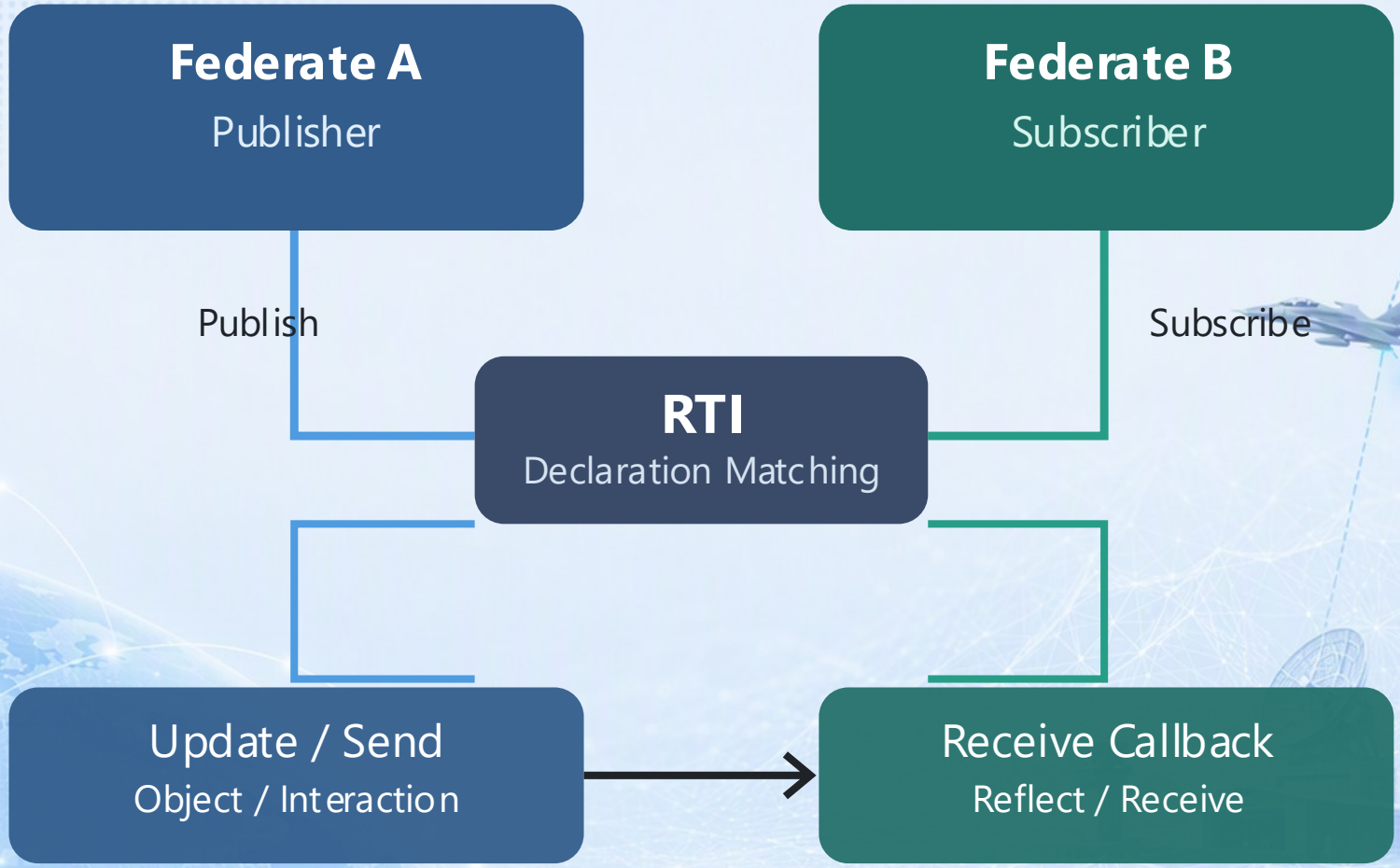

© Zyntelligent Co., Ltd.

# บทที่ 6

# Declaration Management


© Zyntelligent Co., Ltd.

# Declaration Management




Logical data flow through RTI


© Zyntelligent Co., Ltd.

# Declaration Management

การทำงานของ Declaration Management แบ่งเป็น 2 รูปแบบหลัก

| ประเภท | Publish | Subscribe |
|---|---|---|
| Object Class | ประกาศ Attributes ที่ Federate ต้องการเผยแพร่ | เลือก Object Class และ Attributes ที่ต้องการรับ |
| Interaction Class | ประกาศ Interaction Class ที่ต้องการส่ง | เลือก Interaction Class ที่ต้องการรับ |


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction
- Publish Object Class Attributes
- Subscribe Object Class Attributes
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class




© Zyntelligent Co., Ltd.

# Declaration Management

**ObjectClassHandle** เป็น Identifier ที่ RTI ใช้อ้างอิง Object Class ซึ่งกำหนดไว้ใน Federation Object Model (FOM) ผู้ใช้ไม่จำเป็นต้องจัดการ Object Class ด้วยชื่อ String ทุกครั้ง สามารถขอ Handle จาก RTI และนำไปใช้เรียก Service อื่นได้

**การเรียกใช้งาน ObjectClassHandle**

สมมติว่า FOM มี Object Class ชื่อ HLAobjectRoot.Aircraft ซึ่งประกอบด้วย Attributes ได้แก่ Position, Velocity และ Callsign

```
ObjectClassHandle aircraftClassHandle =
    rtiAmbassador.getObjectClassHandle(
        "HLAobjectRoot.Aircraft"
    );
```

ชื่อ Object Class ต้องตรงกับ Hierarchy ที่กำหนดไว้ใน FOM และต้องเรียกหลังจาก Federate เข้าร่วม Federation สำเร็จแล้ว


© Zyntelligent Co., Ltd.

# Declaration Management

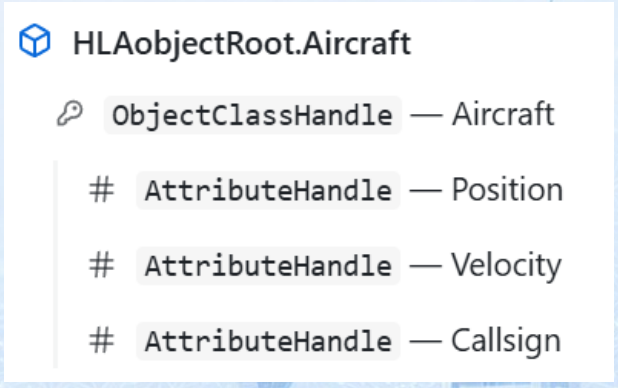
**การเรียกใช้งาน AttributeHandle**

เมื่อได้ ObjectClassHandle แล้ว สามารถเรียก getAttributeHandle() เพื่อรับ Identifier ของ Attribute แต่ละรายการ

```
AttributeHandle positionHandle =
    rtiAmbassador.getAttributeHandle(
        aircraftClassHandle, "Position"
    );

AttributeHandle velocityHandle =
    rtiAmbassador.getAttributeHandle(
        aircraftClassHandle, "Velocity"
    );

AttributeHandle callsignHandle =
    rtiAmbassador.getAttributeHandle(
        aircraftClassHandle, "Callsign"
    );
```





© Zyntelligent Co., Ltd.

# Declaration Management

**การจัดเก็บ Handles**

ควรเก็บ Handles เป็น Fields ภายใน Class เพื่อให้เรียกใช้งานซ้ำได้ โดยไม่จำเป็นต้องเรียกชื่อจาก RTI ทุกครั้ง ช่วยให้โค้ดอ่านง่าย และลดความเสี่ยงจากการพิมพ์ชื่อ Object Class หรือ Attribute ไม่ถูกต้อง

ข้อควรระวัง ไม่ควร Hardcode ตัวเลขของ Handle

```
private ObjectClassHandle aircraftClassHandle;
private AttributeHandle positionHandle;
private AttributeHandle velocityHandle;
private AttributeHandle callsignHandle;
```


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- **การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction**
- Publish Object Class Attributes
- Subscribe Object Class Attributes
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class




© Zyntelligent Co., Ltd.

# Declaration Management

**InteractionClassHandle** ใช้อ้างอิง Interaction Class ซึ่งเป็นรูปแบบของข้อมูลหรือเหตุการณ์ที่ Federate ส่งไปยัง Federate อื่นแตกต่างจาก Object Class ที่ใช้แทน Object Instances และ Attributes โดย Interaction เหมาะกับการส่งเหตุการณ์หรือคำสั่ง เช่น FireWeapon, StartMission หรือ TargetDetected

**การเรียกใช้งาน InteractionClassHandle**

สมมติว่า FOM กำหนด Interaction Class ดังภาพด้านซ้าย สามารถเรียกใช้ Handle ด้วยคำสั่งดังภาพขวา

```
HLAinteractionRoot
└── WeaponFire
    ├── WeaponID
    └── TargetID
```

```
InteractionClassHandle weaponFireHandle =
    rtiAmbassador.getInteractionClassHandle(
        "HLAinteractionRoot.WeaponFire"
    );
```


© Zyntelligent Co., Ltd.

# Declaration Management

**การเรียกใช้งาน ParameterHandle**

Parameter คือข้อมูลที่ส่งมาพร้อม Interaction สามารถเรียก Handle ได้ด้วย

```
ParameterHandle weaponIdHandle =
    rtiAmbassador.getParameterHandle(
        weaponFireHandle, "WeaponID"
    );

ParameterHandle targetIdHandle =
    rtiAmbassador.getParameterHandle(
        weaponFireHandle, "TargetID"
    );
```




© Zyntelligent Co., Ltd.

# Declaration Management

**ความแตกต่างระหว่าง ObjectClassHandle และ InteractionClassHandle**

| คุณสมบัติ | ObjectClassHandle | InteractionClassHandle |
|---|---|---|
| อ้างอิง | Object Class | Interaction Class |
| ข้อมูลประกอบ | Attributes | Parameters |
| การใช้งาน | สถานะของ Object | เหตุการณ์ / คำสั่ง |
| การส่งข้อมูล | updateAttributeValues() | sendInteraction() |
| การรับข้อมูล | reflectAttributeValues() | receiveInteraction() |
| Instance | มี Object Instance | ไม่มี Persistent Object Instance |


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction
- Publish Object Class Attributes
- Subscribe Object Class Attributes
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class




© Zyntelligent Co., Ltd.

# Declaration Management

## Publish Object Class Attributes

Publish เป็นการประกาศต่อ RTI ว่า Federate มีความสามารถและความประสงค์จะเผยแพร่ Attributes ของ Object Class ที่กำหนด

## AttributeHandleSet

ใช้ AttributeHandleSet จัดกลุ่ม Attributes ที่ต้องการ Publish

```
AttributeHandleSet attributes =
    rtiAmbassador.getAttributeHandleSetFactory()
        .create();

attributes.add(positionHandle);
attributes.add(velocityHandle);
attributes.add(callsignHandle);
```

```
rtiAmbassador.publishObjectClassAttributes(
    aircraftClassHandle,
    attributes
);
```

Publish Attributes


© Zyntelligent Co., Ltd.

# Declaration Management

หลังจาก Publish สำเร็จ Federate จึงสามารถดำเนินการ Register Object Instance สำหรับ Object Class ที่ประกาศและส่งข้อมูลของ Attributes ที่ตนมีสิทธิ์อัปเดตได้

```
ObjectInstanceHandle aircraftInstance =
    rtiAmbassador.registerObjectInstance(
        aircraftClassHandle
    );
```

คำสั่งนี้สร้าง Object Instance และคืน ObjectInstanceHandle สำหรับอ้างอิงเครื่องบินที่สร้างขึ้น อย่างไรก็ตามการ Publish และ Register Object Instance จะยังไม่ได้ส่งค่า Attributes ต้องเรียก updateAttributeValues() เพิ่มเติม ซึ่งจะอยู่ในบทของ Object Management


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction
- Publish Object Class Attributes
- **Subscribe Object Class Attributes**
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class




© Zyntelligent Co., Ltd.

# Declaration Management

**Subscribe Object Class Attributes**

Subscribe เป็นการประกาศความสนใจต่อ RTI ว่า Federate ต้องการรับข้อมูล Attributes ใดจาก Object Class ที่กำหนด

ใช้ AttributeHandleSet สำหรับ Subscribe เช่นเดียวกับการ Publish

```
AttributeHandleSet attributes =
    rtiAmbassador.getAttributeHandleSetFactory()
        .create();

attributes.add(positionHandle);
attributes.add(velocityHandle);
```

```
rtiAmbassador.subscribeObjectClassAttributes(
    aircraftClassHandle,
    attributes
);
```

Subscribe Attributes

หลังจาก Subscribe แล้ว RTI จะพิจารณา Object Instances ที่ตรงกับ Subscription และแจ้งข้อมูลผ่าน Callback เมื่อมีเหตุการณ์ที่เกี่ยวข้อง


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction
- Publish Object Class Attributes
- Subscribe Object Class Attributes
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class

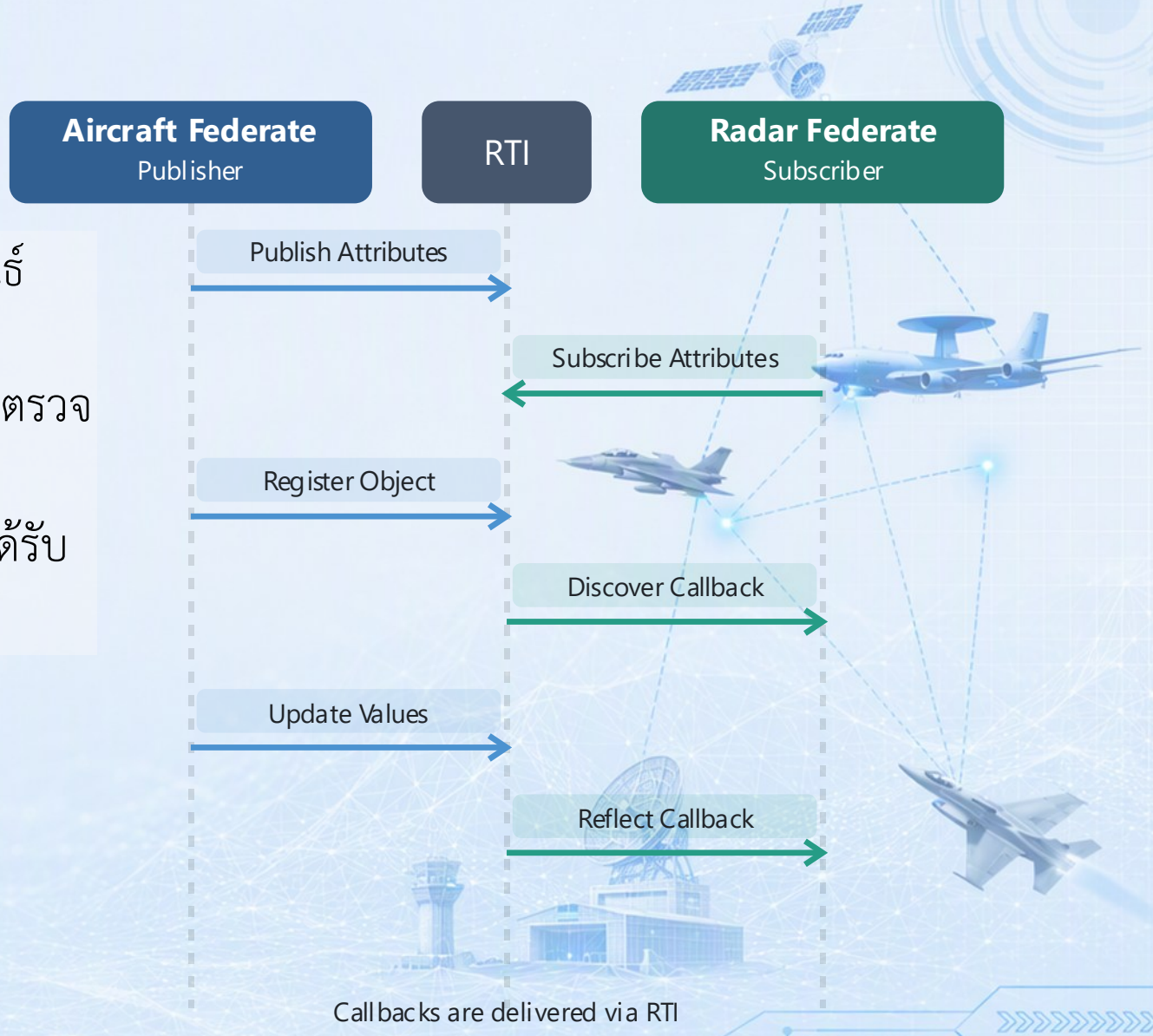



© Zyntelligent Co., Ltd.

# Declaration Management

การ Subscribe Object Class Attributes มีความสัมพันธ์โดยตรงกับ Callback สองรายการสำคัญ ได้แก่

- **discoverObjectInstance()** ใช้แจ้งว่า Federate ตรวจพบ Object Instance ที่เกี่ยวข้อง
- **reflectAttributeValues()** ใช้แจ้งว่า Federate ได้รับค่า Attributes ที่ถูกส่งมาจาก RTI





© Zyntelligent Co., Ltd.

# Declaration Management

**การรับ Discover Object Instance**

ภายใน FederateAmbassador สามารถ Override Callback ดังนี้

```
@Override
public void discoverObjectInstance(
        ObjectInstanceHandle objectHandle,
        ObjectClassHandle classHandle,
        String objectName) {

    System.out.println(
        "Discovered Object: " + objectName
    );
}
```

Callback นี้ทำให้ Subscriber ทราบว่า Object Instance ใดปรากฏอยู่ภายใน Federation

ควรจัดเก็บ **ObjectInstanceHandle** เพื่อใช้อ้างอิง Object ในการรับข้อมูลครั้งต่อไป


© Zyntelligent Co., Ltd.

# Declaration Management

## การรับ Reflect Attribute Values

เมื่อ Publisher ส่ง Attributes ใหม่เข้ามา RTI จะเรียก reflectAttributeValues() ของ Subscriber ที่มี Subscription ตรงกับข้อมูล

```
@Override
public void reflectAttributeValues(
        ObjectInstanceHandle objectHandle,
        AttributeHandleValueMap attributes,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        SupplementalReflectInfo info) {

    System.out.println(
        "Received Attribute Update: " + objectHandle
    );

    for (AttributeHandle handle : attributes.keySet()) {
        byte[] value = attributes.get(handle);

        System.out.println(
            "Attribute: " + handle +
            ", Bytes: " + value.length
        );
    }
}
```

ค่าที่รับได้มีรูปแบบเป็น byte[] จึงต้อง Decode ตาม Data Type ที่กำหนดไว้ใน FOM เช่น HLAfloat64BE, HLAinteger32BE หรือ HLAunicodeString

ในการ Subscribe นั้น RTI ไม่ได้สร้าง Object หรือส่ง Attribute Update ให้ อีกทั้ง Callback ของ Subscriber อาจได้รับข้อมูลบาง Attributes เท่านั้น ขึ้นอยู่กับ Attributes ที่ Publisher ส่งจริงและเงื่อนไขการรับข้อมูล


© Zyntelligent Co., Ltd.

# Declaration Management

- การใช้งานและจัดการ ObjectClassHandle สำหรับ Object Class
- การใช้งานและจัดการ InteractionClassHandle สำหรับ Interaction
- Publish Object Class Attributes
- Subscribe Object Class Attributes
- ความสัมพันธ์ระหว่าง Subscribe กับ Discover / Reflect Callback
- Publish และ Subscribe Interaction Class




© Zyntelligent Co., Ltd.

# Declaration Management

**Publish และ Subscribe Interaction Class**

สำหรับ Interaction Class การประกาศ Publish และ Subscribe จะทำในระดับ Interaction Class โดยไม่จำเป็นต้องเลือก Parameters เป็นรายตัวเหมือน Attributes ของ Object Class

ตัวอย่าง:

- Aircraft Federate ส่ง Interaction WeaponFire
- Command Federate รับ Interaction WeaponFire


© Zyntelligent Co., Ltd.

# Declaration Management

## Publish Interaction Class

```
InteractionClassHandle weaponFireHandle =
    rtiAmbassador.getInteractionClassHandle(
        "HLAinteractionRoot.WeaponFire"
    );

rtiAmbassador.publishInteractionClass(
    weaponFireHandle
);
```

เมื่อ Publish สำเร็จ Federate จึงสามารถส่ง Interaction ผ่าน sendInteraction() ได้

## Subscribe Interaction Class

```
InteractionClassHandle weaponFireHandle =
    rtiAmbassador.getInteractionClassHandle(
        "HLAinteractionRoot.WeaponFire"
    );

rtiAmbassador.subscribeInteractionClass(
    weaponFireHandle
);
```

เมื่อ Interaction ถูกส่งจาก Publisher และตรงกับ Subscription ของ Federate ผู้รับ RTI จะเรียก Callback receiveInteraction()


© Zyntelligent Co., Ltd.

# Declaration Management

## ตัวอย่างการส่ง Interaction

```
ParameterHandleValueMap parameters =
    rtiAmbassador.getParameterHandleValueMapFactory()
        .create(2);

parameters.put(
    weaponIdHandle,
    "AIM-120".getBytes(StandardCharsets.UTF_8)
);

parameters.put(
    targetIdHandle,
    "TARGET-001".getBytes(StandardCharsets.UTF_8)
);

rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null
);
```

## ตัวอย่างการรับ Interaction Callback

```
@Override
public void receiveInteraction(
        InteractionClassHandle interactionClass,
        ParameterHandleValueMap parameters,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        SupplementalReceiveInfo info) {

    System.out.println(
        "Received Interaction: " + interactionClass
    );
}
```


© Zyntelligent Co., Ltd.

# Declaration Management

## เปรียบเทียบการแลกเปลี่ยน Object และ Interaction

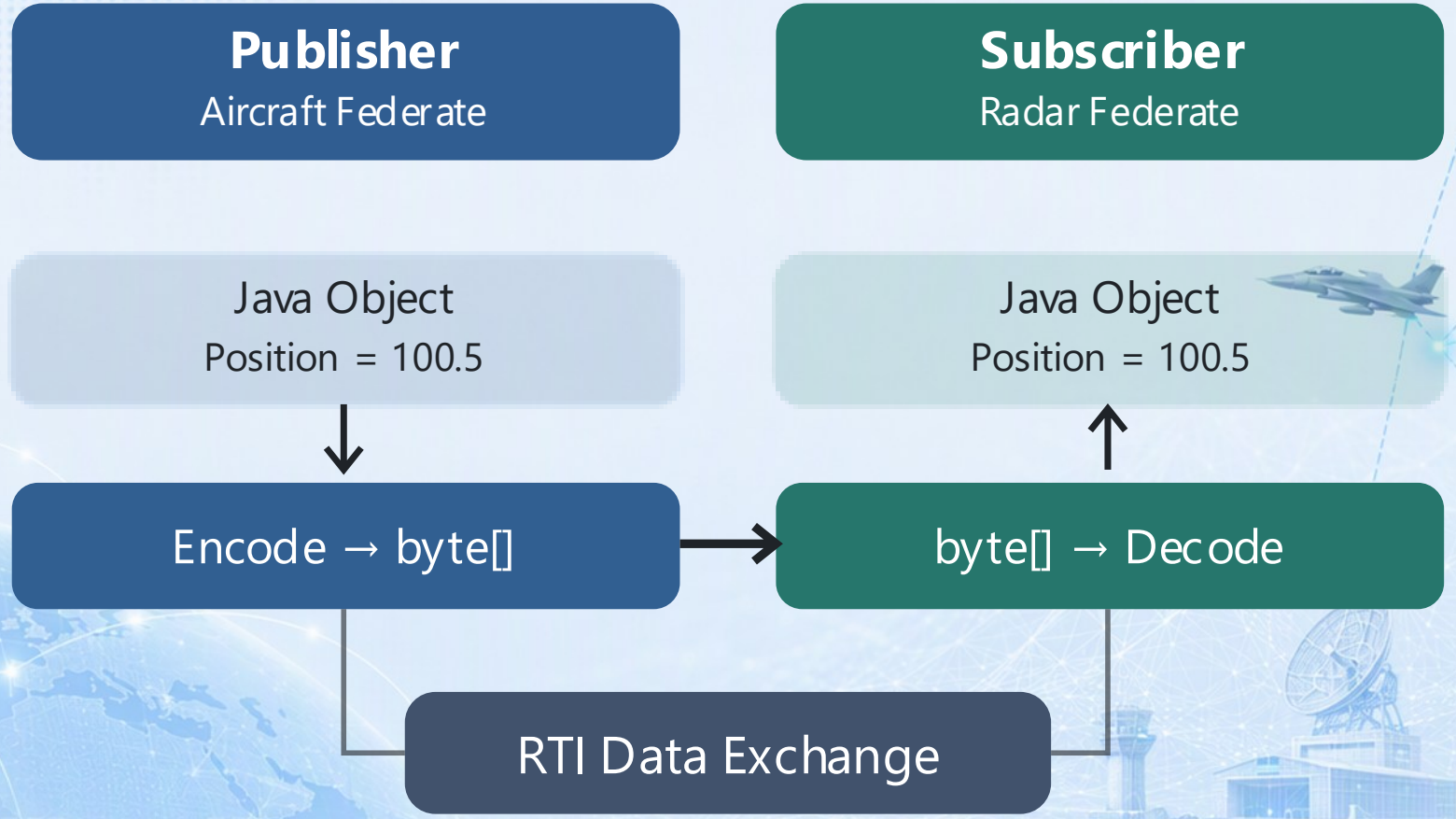
| หัวข้อ | Object Class | Interaction Class |
|---|---|---|
| Publish | Class + Attribute Set | Interaction Class |
| Subscribe | Class + Attribute Set | Interaction Class |
| การส่ง | updateAttributeValues() | sendInteraction() |
| การรับ | reflectAttributeValues() | receiveInteraction() |
| การสร้าง Instance | ต้อง Register | ไม่ต้อง Register |
| รูปแบบข้อมูล | Object State | Event / Message |


© Zyntelligent Co., Ltd.

# บทที่ 7
# Encoding / Decoding ข้อมูล


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล





© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

- การสร้าง Class Encoder / Decoder
- การ Encoding / Decoding ข้อมูล




© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

**การสร้าง Class Encoder / Decoder**

HLA กำหนดรูปแบบการเข้ารหัสข้อมูลผ่าน HLA Data Encoding ตามมาตรฐาน IEEE 1516.2 โดยมีชนิดข้อมูลพื้นฐานและโครงสร้างข้อมูลแบบ Composite

Encoding และ Decoding เป็นกระบวนการสำคัญในการแลกเปลี่ยนข้อมูลระหว่าง Federates ภายใน Federation โดยมีหน้าที่แปลงข้อมูลจากชนิดข้อมูลที่ใช้ในโปรแกรม เช่น int, double, String หรือ Object ให้กลายเป็นข้อมูลไบนารี (byte[]) เพื่อส่งผ่าน RTI และแปลงกลับเป็นข้อมูลต้นฉบับเมื่อได้รับจาก Federate อื่น


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

| HLA Data Type | Java Type | รายละเอียด |
|---|---|---|
| HLAinteger16BE | short | Integer 16-bit, Big-endian |
| HLAinteger32BE | int | Integer 32-bit, Big-endian |
| HLAinteger64BE | long | Integer 64-bit, Big-endian |
| HLAfloat32BE | float | Floating-point 32-bit |
| HLAfloat64BE | double | Floating-point 64-bit |
| HLAboolean | boolean | Boolean |
| HLAunicodeString | String | Unicode String |
| HLAfixedRecord | Composite Object | ชุดข้อมูลที่มีโครงสร้างตายตัว |
| HLAfixedArray | Array | Array ขนาดคงที่ |
| HLAvariableArray | Array / List | Array ขนาดเปลี่ยนแปลงได้ |


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

**การสร้าง Class Encoder / Decoder**

HLA กำหนดรูปแบบการเข้ารหัสข้อมูลผ่าน HLA Data Encoding ตามมาตรฐาน IEEE 1516.2 โดยมีชนิดข้อมูลพื้นฐานและโครงสร้างข้อมูลแบบ Composite

Encoding และ Decoding เป็นกระบวนการสำคัญในการแลกเปลี่ยนข้อมูลระหว่าง Federates ภายใน Federation โดยมีหน้าที่แปลงข้อมูลจากชนิดข้อมูลที่ใช้ในโปรแกรม เช่น int, double, String หรือ Object ให้กลายเป็นข้อมูลไบนารี (byte[]) เพื่อส่งผ่าน RTI และแปลงกลับเป็นข้อมูลต้นฉบับเมื่อได้รับจาก Federate อื่น


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

## การสร้าง Class สำหรับ Encoding / Decoding

```
import hla.rti1516e.*;
import hla.rti1516e.encoding.*;

EncoderFactory encoderFactory =
    RtiFactoryFactory.getRtiFactory()
        .getEncoderFactory();
```

Factory นี้สามารถใช้สร้าง HLAfloat64BE, HLAinteger32BE, HLAunicodeString และ Data Elements อื่น ๆ

```
// Encode double
public byte[] encodeDouble(double value)
        throws EncoderException {

    HLAfloat64BE element =
            factory.createHLAfloat64BE(value);

    return element.toByteArray();
}

// Decode double
public double decodeDouble(byte[] bytes)
        throws DecoderException {

    HLAfloat64BE element =
            factory.createHLAfloat64BE();

    element.decode(bytes);
    return element.getValue();
}
```

จากตัวอย่าง Methods นี้มีหน้าที่หลัก 2 ส่วน

**Encoder**: รับข้อมูลจาก Java แล้วแปลงเป็น byte[] ผ่าน toByteArray()

**Decoder**: รับ byte[] แล้วเรียก decode() และ getValue() เพื่อแปลงเป็นข้อมูล Java


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

**การสร้าง Encoder / Decoder สำหรับข้อมูลแบบ Composite**

หาก FOM กำหนด Attribute Position เป็นโครงสร้างข้อมูลที่ประกอบด้วย Latitude, Longitude และ Altitude สามารถใช้ HLAfixedRecord ได้

```
PositionStruct
├── Latitude  : HLAfloat64BE
├── Longitude : HLAfloat64BE
└── Altitude  : HLAfloat64BE
```

```
public record Position(
    double latitude,
    double longitude,
    double altitude
) {}
```


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

## ตัวอย่าง Method สำหรับ Encode

```
public byte[] encodePosition(Position position)
        throws EncoderException {

    HLAfixedRecord record =
            factory.createHLAfixedRecord();

    record.add(
        factory.createHLAfloat64BE(position.latitude())
    );

    record.add(
        factory.createHLAfloat64BE(position.longitude())
    );

    record.add(
        factory.createHLAfloat64BE(position.altitude())
    );

    return record.toByteArray();
}
```

## ตัวอย่าง Method สำหรับ Decode

```
public Position decodePosition(byte[] bytes)
        throws DecoderException {

    HLAfloat64BE latitude =
            factory.createHLAfloat64BE();

    HLAfloat64BE longitude =
            factory.createHLAfloat64BE();

    HLAfloat64BE altitude =
            factory.createHLAfloat64BE();

    HLAfixedRecord record =
            factory.createHLAfixedRecord();

    record.add(latitude);
    record.add(longitude);
    record.add(altitude);

    record.decode(bytes);

    return new Position(
        latitude.getValue(),
        longitude.getValue(),
        altitude.getValue()
    );
}
```



ในกรณีนี้ลำดับ Fields ที่เพิ่มเข้า HLAfixedRecord ต้องตรงกับที่กำหนดใน FOM เพราะลำดับมีผลโดยตรงต่อการ Encode/Decode


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

- การสร้าง Class Encoder / Decoder
- การ Encoding / Decoding ข้อมูล




© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

**การ Encoding ข้อมูลก่อนส่งผ่าน RTI**

ตัวอย่าง Aircraft Federate ต้องการส่งข้อมูลดังนี้

| Attribute | HLA Data Type | ตัวอย่างค่า |
|---|---|---|
| Position | PositionStruct | 16.44, 102.83, 1500.0 |
| Velocity | HLAfloat64BE | 250.5 |
| Callsign | HLAunicodeString | FALCON01 |


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

เตรียม Attributes

```
HlaDataCodec codec = new HlaDataCodec(encoderFactory);

AttributeHandleValueMap attributes = rtiAmbassador
                    .getAttributeHandleValueMapFactory().create(3);

Position position = new Position(16.44, 102.83, 1500.0);

attributes.put(
    positionHandle,
    codec.encodePosition(position)
);

attributes.put(
    velocityHandle,
    codec.encodeDouble(250.5)
);

attributes.put(
    callsignHandle,
    codec.encodeString("FALCON01")
);
```

เมื่อเตรียม Attributes เรียบร้อยแล้ว สามารถส่งข้อมูลผ่าน RTI ได้

```
rtiAmbassador.updateAttributeValues(
    aircraftInstance,
    attributes,
    null
);
```

โดย aircraftInstance คือ ObjectInstanceHandle ที่ได้จาก registerObjectInstance() ซึ่ง Federate ต้อง Publish Attributes และมีสิทธิ์อัปเดตข้อมูลก่อนส่ง


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

**การ Decoding ข้อมูลก่อนส่งผ่าน RTI**

เมื่อ Subscriber ได้รับ Attribute Updates ผ่าน Callback reflectAttributeValues() ต้องนำข้อมูล byte[] มาถอดรหัสตามชนิดข้อมูลใน FOM


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

```java
@Override
public void reflectAttributeValues(
        ObjectInstanceHandle objectHandle,
        AttributeHandleValueMap attributes,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        SupplementalReflectInfo info) {

    try {
        if (attributes.containsKey(positionHandle)) {

            Position position = codec.decodePosition(
                attributes.get(positionHandle)
            );
            System.out.println("Position: " + position);
        }

        if (attributes.containsKey(velocityHandle)) {

            double velocity = codec.decodeDouble(
                attributes.get(velocityHandle)
            );
            System.out.println("Velocity: " + velocity);
        }

        if (attributes.containsKey(callsignHandle)) {

            String callsign = codec.decodeString(
                attributes.get(callsignHandle)
            );
            System.out.println("Callsign: " + callsign);
        }

    } catch (DecoderException e) {
        System.err.println("Failed to decode attributes: " + e.getMessage());
    }
}
```

reflectAttributeValues() อาจได้รับเพียงบาง Attributes
จึงต้องตรวจสอบ containsKey() ก่อน Decode
ทุก Callback อาจจะไม่มีข้อมูลครบทุก Field

# Encoding / Decoding ข้อมูล

## การ Encoding / Decoding Interaction Parameters

การ Encode/Decode สำหรับ Interaction ใช้แนวคิดเดียวกับ Object Attributes แต่เก็บข้อมูลใน ParameterHandleValueMap

```
ParameterHandleValueMap parameters =
    rtiAmbassador.getParameterHandleValueMapFactory()
        .create(2);

parameters.put(
    weaponIdHandle,
    codec.encodeString("AIM-120")
);

parameters.put(
    targetIdHandle,
    codec.encodeInteger(1001)
);

rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null
);
```

ตัวอย่างการ Encoding และส่ง Interaction

```
HLAinteractionRoot.WeaponFire
├── WeaponID : HLAunicodeString
└── TargetID : HLAinteger32BE
```

```
if (interactionClass.equals(weaponFireHandle)) {

    String weaponId = codec.decodeString(
        parameters.get(weaponIdHandle)
    );

    int targetId = codec.decodeInteger(
        parameters.get(targetIdHandle)
    );

    System.out.println("Weapon: " + weaponId);
    System.out.println("Target: " + targetId);
}
```

ตัวอย่างการ Decoding และส่ง Interaction


© Zyntelligent Co., Ltd.

# Encoding / Decoding ข้อมูล

## การ Encoding / Decoding Interaction Parameters

การ Encode/Decode สำหรับ Interaction ใช้แนวคิดเดียวกับ Object Attributes แต่เก็บข้อมูลใน ParameterHandleValueMap

```
ParameterHandleValueMap parameters =
    rtiAmbassador.getParameterHandleValueMapFactory()
        .create(2);

parameters.put(
    weaponIdHandle,
    codec.encodeString("AIM-120")
);

parameters.put(
    targetIdHandle,
    codec.encodeInteger(1001)
);

rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null
);
```

ตัวอย่างการ Encoding และส่ง Interaction

```
HLAinteractionRoot.WeaponFire
├── WeaponID : HLAunicodeString
└── TargetID : HLAinteger32BE
```

```
if (interactionClass.equals(weaponFireHandle)) {

    String weaponId = codec.decodeString(
        parameters.get(weaponIdHandle)
    );

    int targetId = codec.decodeInteger(
        parameters.get(targetIdHandle)
    );

    System.out.println("Weapon: " + weaponId);
    System.out.println("Target: " + targetId);
}
```

ตัวอย่างการ Decoding และส่ง Interaction


© Zyntelligent Co., Ltd.

# บทที่ 8
# Object Instance Register, Discover และ Remove Object


© Zyntelligent Co., Ltd.

# Object Instance Register, Discover และ Remove Object

- Register Object Instance และกำหนดชื่อ Instance
- การ discover object Instance
- การรับ removeObjectInstance เมื่อ Object ถูกลบ




© Zyntelligent Co., Ltd.

# Object Instance Register, Discover และ Remove Object

## Register Object Instance

```
ObjectInstanceHandle aircraftInstance =
    rtiAmbassador.registerObjectInstance(
        aircraftClassHandle
    );
```

callback

## Discover Object Instance

```
@Override
public void discoverObjectInstance(
        ObjectInstanceHandle objectHandle,
        ObjectClassHandle classHandle,
        String objectName) {

    System.out.println("Discovered: " + objectName);

    objects.put(objectHandle, objectName);
}
```

## Remove Object Instance

```
rtiAmbassador.deleteObjectInstance(
    aircraftInstance,
    null
);
```

callback

## Discover Object Instance

```
@Override
public void removeObjectInstance(
        ObjectInstanceHandle objectHandle,
        byte[] tag,
        OrderType sentOrder,
        SupplementalRemoveInfo info) {

    objects.remove(objectHandle);

    System.out.println("Object removed: " + objectHandle);
}
```


© Zyntelligent Co., Ltd.

# บทที่ 9
# Interaction Publish, Subscribe, Send และ Receive


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

- Interaction และ Parameter Handles
- สร้าง ParameterHandleValueMap และ Encode Parameters
- ส่ง Interaction ด้วย sendInteraction()
- รับ receiveInteraction() ทั้ง Receive Order และ Timestamp Order


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

## Interaction / Parameter Handle

```
InteractionClassHandle weaponFireHandle =
    rtiAmbassador.getInteractionClassHandle(
        "HLAinteractionRoot.WeaponFire"
    );

ParameterHandle weaponIdHandle =
    rtiAmbassador.getParameterHandle(
        weaponFireHandle, "WeaponID"
    );

ParameterHandle targetIdHandle =
    rtiAmbassador.getParameterHandle(
        weaponFireHandle, "TargetID"
    );
```

## Publish/ Subscribe Interaction Class

```
rtiAmbassador.publishInteractionClass(
    weaponFireHandle
);

rtiAmbassador.subscribeInteractionClass(
    weaponFireHandle
);
```


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

- Interaction และ Parameter Handles
- สร้าง ParameterHandleValueMap และ Encode Parameters
- ส่ง Interaction ด้วย sendInteraction()
- รับ receiveInteraction() ทั้ง Receive Order และ Timestamp Order


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

**สร้าง ParameterHandleValueMap และ Encode Parameters**

ใช้ ParameterHandleValueMap สำหรับจัดเก็บข้อมูล Parameters ที่ต้องการส่ง โดยนำค่ามา Encode เป็น byte[] ตาม Data Type ที่กำหนดใน FOM

```
ParameterHandleValueMap parameters =
    rtiAmbassador.getParameterHandleValueMapFactory()
        .create(2);

parameters.put(
    weaponIdHandle,
    "AIM-120".getBytes(StandardCharsets.UTF_8)
);

parameters.put(
    targetIdHandle,
    "TARGET-001".getBytes(StandardCharsets.UTF_8)
);

rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null
);
```


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

- Interaction และ Parameter Handles
- สร้าง ParameterHandleValueMap และ Encode Parameters
- ส่ง Interaction ด้วย sendInteraction()
- รับ receiveInteraction() ทั้ง Receive Order และ Timestamp Order


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

## ส่ง Interaction ด้วย sendInteraction()

ใช้ sendInteraction() เพื่อส่ง Interaction ไปยัง RTI ซึ่งจะจัดส่งไปยัง Federates ที่ Subscribe และอยู่ภายใต้เงื่อนไขการรับข้อมูล

### ส่งแบบ Receive Order (RO)

```
rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null
);
```

### ส่งแบบ Timestamp Order (TSO)

```
import hla.rti1516e.time.HLAfloat64Time;

HLAfloat64Time sendTime = new HLAfloat64Time(10.0);

rtiAmbassador.sendInteraction(
    weaponFireHandle,
    parameters,
    null,
    sendTime
);
```

*การส่งแบบ TSO ระบุ Logical Time ของเหตุการณ์ และต้องตั้งค่า Time Management ให้ถูกต้อง (อยู่ในบท Time Management)


© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

- Interaction และ Parameter Handles
- สร้าง ParameterHandleValueMap และ Encode Parameters
- ส่ง Interaction ด้วย sendInteraction()
- รับ receiveInteraction() ทั้ง Receive Order และ Timestamp Order




© Zyntelligent Co., Ltd.

# Interaction Publish, Subscribe, Send และ Receive

## รับ receiveInteraction() ทั้ง Receive Order และ Timestamp Order

เมื่อ Subscriber ได้รับ Interaction จาก RTI ระบบจะเรียก receiveInteraction() ภายใน Federate Ambassador

### รับแบบ Receive Order (RO)

```
@Override
public void receiveInteraction(
        InteractionClassHandle interactionClass,
        ParameterHandleValueMap parameters,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        SupplementalReceiveInfo info) {

    System.out.println("Received Interaction (RO)");
}
```

### รับแบบ Timestamp Order (TSO)

```
@Override
public void receiveInteraction(
        InteractionClassHandle interactionClass,
        ParameterHandleValueMap parameters,
        byte[] tag,
        OrderType sentOrder,
        TransportationTypeHandle transport,
        LogicalTime time,
        OrderType receivedOrder,
        MessageRetractionHandle retractionHandle,
        SupplementalReceiveInfo info) {

    System.out.println("Received Interaction");
    System.out.println("Time: " + time);
    System.out.println("Order: " + receivedOrder);
}
```

*ควรตรวจสอบ receivedOrder เพื่อทราบลำดับการรับ เนื่องจาก Interaction ที่ถูกส่งพร้อม Timestamp ไม่จำเป็นต้องถูกส่งแบบ TSO เสมอไป ขึ้นอยู่กับการตั้งค่า Time Management และ Ordering ที่เกี่ยวข้อง


© Zyntelligent Co., Ltd.




© Zyntelligent Co., Ltd.